Refinement and Inference Growth in SKJS: a Soundness Model of the Type Checker

Models revision `cf5a6bc7` of `main` (2026-07-09)

Abstract

We formalize the internal type representation used by SKJS and prove soundness properties for its two flow-sensitive type transformations: predicate-driven *refinement* (control-flow narrowing, `evalCond`) and *constant removal* (the residue of the former general type-difference operation). We model the inference substitution as a small state machine and state the invariants under which inference growth (`growTVars`) cannot cause unsound acceptance. Everything the model does not describe concretely is either an explicit abstract interface with stated axioms (collected in Section 8, Assumptions and Proof Obligations) or explicitly out of scope with a justification that the omission cannot hide a soundness hole (Section 1). This document supersedes the draft attached to PR #342 ("Refinement and Type Difference in SKJS").

Contents

1 Scope, Method, and Modeling Contract **2**

2 Runtime Model **3**

3 Internal Type Representation **5**
- 3.1 Simple types . . . 5
- 3.2 Object branches and fields . . . 6
- 3.3 The union spine . . . 7
- 3.4 Operation interfaces . . . 8

4 Refinement (Control-Flow Narrowing) **9**
- 4.1 The single refinement primitive . . . 9
- 4.2 Environments, evaluation, and branch soundness . . . 10
- 4.3 Master list . . . 11
- 4.4 Rules and soundness . . . 11
  - 4.4.1 R-Eq: strict equality on a variable . . . 11
  - 4.4.2 R-EqUndef: comparison with `undefined` . . . 12
  - 4.4.3 R-EqField: strict equality on a field path . . . 12
  - 4.4.4 R-Typeof . . . 13
  - 4.4.5 R-IsArray . . . 14
  - 4.4.6 R-Inst: `instanceof` . . . 14
  - 4.4.7 R-In . . . 15
  - 4.4.8 R-Not, R-And, R-Or . . . 16
  - 4.4.9 R-Truthy, R-TruthyField . . . 16

5 Constant Removal **17**

6 Inference: Substitution Growth and Validation **18**
6.1 The substitution state machine 18
6.2 Growth as pre-seeding; validation decides 20

7 Worked Examples **21**
7.1 Literal narrowing 21
7.2 Persisting incompleteness: subtracting from top 21
7.3 Persisting incompleteness: no false-branch `instanceof` 21
7.4 Persisting incompleteness: multi-literal `in` keys 21
7.5 Known unsoundness: string-kind falsiness 21

8 Assumptions and Proof Obligations **22**

A Implementation Correspondence **34**

B Maintenance **34**

1 Scope, Method, and Modeling Contract

Pinned revision. All implementation references (file, class, function) are against commit `cf5a6bc7`. The document is maintained by the discipline of Appendix B: every inference rule in the refinement master list corresponds to one dispatch arm of `evalCond`, and drift between the list and the dispatch is intended to be mechanically checkable. Two terms of art for that mechanism, used throughout: an *implementation anchor* is a file/class/function reference (typeset as impl: `...`) locating the code a definition or rule models — the maintenance check verifies anchors bidirectionally; a *pinned test* is a test in the repository that fails when a documented behavior changes, forcing the corresponding passage to be updated in the same change.

Modeling contract. Every behavior of the implementation that can affect type soundness is placed in exactly one of three bins:

1. **Modeled**: defined concretely in this document and proved sound.
2. **Abstracted**: hidden behind an interface with explicitly stated axioms. All such axioms are collected in Section 8 (Assumptions and Proof Obligations), each with an implementation anchor and a status from the fixed vocabulary PROVED / ASSUMED / OPEN / VIOLATED defined there.
3. **Out of scope**: listed below, each with a reason why omission cannot create a hole in the soundness argument.

An implementation behavior that fits none of the bins is a defect of this document.

Terminology. The model deliberately keeps the implementation's names so that every term can be grepped for; the cost is a few SKJS-specific usages, fixed here. An *object branch* is one alternative of an object type's internal union (the implementation's `TObject.branches`); it is always qualified, since *true/false branch* refers to control flow. A *member* is one SType alternative of an STypes union; a *clause* pairs type parameters with such a union. A *candidate* is an ordinary

inference variable (`TVID`); *grow* and *seal* are the substitution transitions of Section 6; *expansion* rewrites a type to its instance-plus-clauses view under the current substitution. A *brand* is the `Array.isArray` marker on an object branch; a *phantom field* records runtime-confirmed presence without a declaration. "Open" is used only for the substitution state SVOpen and the obligation status; the object semantics of Definition 13 is called *extensible*.

Out of scope, with justification.

- *Syntax rejected at normalization.* Conditional types (`T extends U ? X : Y`, SKJS9020), `infer` (9017), template literal types (9018), indexed access `T[K]` (9021), type predicates `x is T` (9015), `asserts` predicates (9013): these produce a fatal diagnostic in `TEnv::normalize` and never reach the type machinery, so they are outside the soundness surface by construction.

  impl: `TEnv.sk / normalize`

- *Unhandled condition forms.* The refinement dispatch falls through to the identity on any condition shape it does not recognize (Rule R-OTHER), and the identity refinement is trivially sound (Lemma 24). Consequently a stale or incomplete master list makes this document *incomplete*, never *unsound*, as a description of narrowing.

- *Mutation during checking of straight-line code.* This document proves *one-step branch soundness*: a single condition is evaluated against a fixed store. Assignment, loops, and join points are treated only through the union-soundness of scope merging (Lemma 35); a full small-step preservation theorem is future work. The interaction of narrowing with mutation *during condition evaluation* is governed by assumptions **L-PURE** and **L-LOCAL**.

- *Flow-typing sites outside condition narrowing.* The master list covers condition forms only. The checker performs flow-sensitive type updates at five further sites, listed here so the boundary is a stated enumeration rather than an implication: assignment to a local updates its scope entry (`TypeCheckExpr.sk`, `Var`-LHS `scope.set`); literal-typed local objects grow fields on assignment; `this` is re-typed after `super(...)`; the non-null assertion `x!` and optional chaining remove nullish members from the *expression's* type (no scope mutation, via `removeUndefined`); and the `for-in` index mechanism suppresses element errors without narrowing. None of these is a condition refinement; each would be a case of the future preservation theorem. Note also that `for`-loop conditions do *not* narrow.

- *Numeric coercions, operators, and standard-library semantics.* Only the condition forms of the master list are given evaluation semantics.

2 Runtime Model

Definition 1 (Runtime Tags)**.** Tags *is the nine-element set*

$$\mathsf{Tags} = \{\mathsf{undef}, \mathsf{void}, \mathsf{null}, \mathsf{bool}, \mathsf{num}, \mathsf{str}, \mathsf{bigint}, \mathsf{sym}, \mathsf{obj}\}.$$

impl: `TType.sk / STypeTag (TagUndefined ... TagObject)`

There is no dedicated tag for tuples, promises, or algebraic data types: tuples and promises are object-tagged values whose types live in the instance layer or as object types (`SKJS_Tuple`/`ReadonlyTuple` instances; `Promise` as a stub-declared object type), and discriminated unions are ordinary object types with discriminant fields.

Definition 2 (Runtime Values)**.** *Fix payload sets* $(\mathsf{Payload}_k)_{k\in\mathsf{Tags}}$ *and define* $\mathsf{Val} \triangleq \{(k,p) \mid k \in \mathsf{Tags},\ p \in \mathsf{Payload}_k\}$*, with* $\mathsf{rtag}((k,p)) = k$*. Primitive payloads are concrete; object payloads are abstract:*

$$\mathsf{Payload}_{\mathsf{undef}} = \{\star\} \qquad \mathsf{Payload}_{\mathsf{void}} = \{\star\} \qquad \mathsf{Payload}_{\mathsf{null}} = \{\star\}$$
$$\mathsf{Payload}_{\mathsf{bool}} = \{\mathsf{true},\mathsf{false}\} \qquad \mathsf{Payload}_{\mathsf{num}} = \mathbb{F} \qquad \mathsf{Payload}_{\mathsf{str}} = \mathsf{String}$$
$$\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z} \qquad \mathsf{Payload}_{\mathsf{sym}} = \mathsf{SymVal} \qquad \mathsf{Payload}_{\mathsf{obj}} = \mathsf{Payload}'_{\mathsf{obj}}$$

where $\mathbb{F}$ *is the set of IEEE-754 doubles (including* -0 *and* NaN*),* SymVal *an abstract infinite set of symbol identities, and* $\mathsf{Payload}'_{\mathsf{obj}}$ *an arbitrary set. The tag* void *models the checker's distinct* `void` *type; at runtime its unique payload is the value* `undefined`*, which is why refinements treat the tags* undef *and* void *together (Definition 26).*

Definition 3 (Object Interface)**.** *Let* $\mathsf{ObjVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = \mathsf{obj}\}$*. We assume the following abstract interface on object values:*

- *Keys:* $\mathsf{Key} \triangleq \mathsf{String} \uplus \mathsf{SymVal}$*.*
- $\mathsf{props} : \mathsf{ObjVal} \to \mathcal{P}(\mathsf{Key})$ *and a partial* $\mathsf{get} : \mathsf{ObjVal} \times \mathsf{Key} \rightharpoonup \mathsf{Val}$*, with* $\mathsf{get}(o,p)$ *defined iff* $p \in \mathsf{props}(o)$*. Write* $\mathsf{has}(o,p)$ *for* $p \in \mathsf{props}(o)$*.*
- $\mathsf{callable}, \mathsf{constructible} : \mathsf{ObjVal} \to \{\mathsf{true},\mathsf{false}\}$*, with the JS fact* $\mathsf{constructible}(o) \implies \mathsf{callable}(o)$ *(every constructor is a function;* `typeof` *yields* `"function"` *for it).*
- $\mathsf{isArrayV} : \mathsf{ObjVal} \to \{\mathsf{true},\mathsf{false}\}$*, the runtime truth of* `Array.isArray`*.*
- *A classification* $\mathsf{kind} : \mathsf{String} \to \{\mathsf{strKey},\mathsf{numKey}\}$ *of string keys into numeric and non-numeric spellings, used by index signatures.*

This interface is a snapshot view: get *describes the properties of* o *at the moment a condition is evaluated. See assumption* ***L-PURE****.*

Definition 4 (Strict Equality)**.** *JS strict equality is* not *identity on* Val*; we model it as a relation* $\approx \subseteq \mathsf{Val} \times \mathsf{Val}$*:* $v \approx w$ *iff one of*

- $\mathsf{rtag}(v), \mathsf{rtag}(w) \in \{\mathsf{undef},\mathsf{void}\}$ *(the* `void` *tag is a checker fiction for the runtime value* `undefined`*);*
- $\mathsf{rtag}(v) = \mathsf{rtag}(w) = \mathsf{num}$ *and the payloads are equal as IEEE doubles with* -0 *identified with* 0 *and* NaN *equal to nothing (including itself);*
- $\mathsf{rtag}(v) = \mathsf{rtag}(w) \notin \{\mathsf{undef},\mathsf{void},\mathsf{num}\}$ *and the payloads are equal (reference identity for objects).*

Note $\approx$ *is not reflexive (*$\mathsf{NaN} \not\approx \mathsf{NaN}$*) and crosses the* undef/void *tag pair. A set* $X \subseteq \mathsf{Val}$ *is* $\approx$-closed *if* $v \in X$ *and* $w \approx v$ *imply* $w \in X$*.*

Lemma 5 ($\approx$ is a Partial Equivalence)**.** $\approx$ *is symmetric and transitive; its domain (the values related to themselves) is* Val *minus the* NaN*-payload numbers. On that domain "the* $\approx$*-class of* v*" is well-defined.*

Proof. Each arm is symmetric. Transitivity: a chain stays within the $\{\mathsf{undef},\mathsf{void}\}$ pair or within a single tag with payload equality (for numbers, equality-with-$\pm$0-identified is transitive and NaN relates to nothing, so chains through it are vacuous). □

Definition 6 (Truthiness)**.** truthy : Val $\to$ {true, false} *is defined per tag: values of tags* undef, void, null *are falsy;* (bool, b) *is truthy iff* $b =$ true*;* (num, n) *is falsy iff* $n \in \{0, -0, \mathsf{NaN}\}$*;* (str, s) *is falsy iff* $s =$ `""`*;* (bigint, z) *is falsy iff* $z = 0$*; symbols and objects are always truthy.*
impl: `TType.sk / TPrimValueKind::isFalsy, SType::falsiness`

3 Internal Type Representation

3.1 Simple types

Definition 7 (Simple Types)**.** SType *has exactly three constructor families:*

- unit(u) *for* $u \in \{$***`undefined`***, ***`void`***, ***`null`***$\}$*;*
- prim(k, L) *for* $k \in \{$***`bool`***, ***`num`***, ***`str`***, ***`bigint`***, ***`sym`***$\}$ *and* $L \in \{\top\} \cup \mathcal{P}^{+}_{\text{fin}}(\mathsf{Lit}_k)$*, where* Lit_k *is the set of literal spellings of kind* k *and* $\top$ *means the full primitive;*
- object(B) *for a nonempty finite set* B *of* branches *(Definition 11).*

tag *maps each constructor to the evident tag.*
impl: `TType.sk / TPrimUnit, TPrimValue, TObject; SType::tag`

Definition 8 (Literal Denotation)**.** *Assume a denotation* $\delta_k : \mathsf{Lit}_k \to \mathsf{Payload}_{\mathsf{tag}(k)}$ *from literal spellings to payloads, and define*

$$\begin{aligned}
[\![\mathsf{unit}(u)]\!] &\triangleq \{(\mathsf{tag}(u), \star)\} \\
[\![\mathsf{prim}(k, \top)]\!] &\triangleq \{(\mathsf{tag}(k), p) \mid p \in \mathsf{Payload}_{\mathsf{tag}(k)}\} \\
[\![\mathsf{prim}(k, L)]\!] &\triangleq \{\, v \mid \mathsf{rtag}(v) = \mathsf{tag}(k),\ \exists \ell \in L.\ v \approx (\mathsf{tag}(k), \delta_k(\ell)) \,\}.
\end{aligned}$$

Literal-type denotations are the same-tag $\approx$-closure *of their spellings' denotations: the literal type* `0` *contains both* 0 *and* -0*, matching JS strict equality and tsc's treatment of numeric literal types. (For non-numeric kinds the closure is the identity. The cross-tag* undef/void *arm of* $\approx$ *is deliberately* not *baked into unit denotations; the rules that compare against* ***`undefined`*** *handle the two tags jointly, and the residual corner is assumption* ***L-SEQ****.) We assume* Lit_{num} *contains no* NaN *spelling: there is no NaN literal type, and a hypothetical one would denote* $\emptyset$ *under closure semantics (nothing is* $\approx \mathsf{NaN}$*), making* ***L-TC*** *unsatisfiable for it. The implementation stores literal* spellings *and compares them syntactically; assumption* ***L-LIT*** *states the assumptions this requires. Unique symbols are represented as literals of kind* ***`sym`*** *whose stored string is a* synthesized position-derived identity *(****`unique_symbol:file:sym:offset:name`****), not a source lexeme — so distinct unique symbols have distinct denotations by construction.*
impl: `PrimLiterals.sk; TType.sk / TPrimValue; TEnv.sk / uniqueSymbol`

Lemma 9 (A0: Tag Consistency)**.** $\forall s \in \mathsf{SType},\ v \in [\![s]\!].\ \mathsf{rtag}(v) = \mathsf{tag}(s)$.

Proof. Immediate from the definitions above and Definition 13, whose members are drawn from ObjVal. □

Lemma 10 (A1: Tag Disjointness)**.** $\forall s_1, s_2 \in \mathsf{SType}.\ \mathsf{tag}(s_1) \neq \mathsf{tag}(s_2) \implies [\![s_1]\!] \cap [\![s_2]\!] = \emptyset$.

Proof. By A0, as values carry a unique tag. □

3.2 Object branches and fields

Definition 11 (Fields and Branches)**.** *A* field *is a record* $f = (\mathsf{name}, \mathsf{opt}, \mathsf{get}, \mathsf{set}, \mathsf{access})$ *with* $\mathsf{name} \in \mathsf{Key}$*,* $\mathsf{opt} \in \{\mathsf{true}, \mathsf{false}\}$*,* $\mathsf{get}, \mathsf{set} \in \mathsf{TType}$*, and* $\mathsf{access} \in \{\mathsf{Pub}, \mathsf{Priv}(\mathit{owner}), \mathsf{Never}, \mathsf{Phantom}\}$*. An* object branch $b \in \mathsf{TFields}$ *is a record consisting of:*

- *finite maps* $\mathsf{flds}, \mathsf{symflds}$ *from string/symbol keys to fields;*
- *optional index fields* idx_κ *for* $\kappa \in \{\mathsf{strKey}, \mathsf{numKey}, \mathsf{symKey}\}$*, each a field;*
- $\mathsf{call}, \mathsf{new} \in \{\mathsf{None}\} \cup \{(\mathit{opt}, \sigma) \mid \mathit{opt} \in \{\mathsf{true}, \mathsf{false}\}\}$ *where* σ *ranges over abstract function signatures;*
- $\mathsf{isArr} \in \{\mathsf{None}, \mathsf{Some}(\mathsf{true}), \mathsf{Some}(\mathsf{false})\}$*, the array brand.*

Readonly fields are encoded as $\mathsf{set} = \bot$ *(`TField::create` with `isReadonly`). The implementation's `TObject` additionally carries `unexpandedType : ?TSInst`, a best-effort record of the named type an object came from; it is a printing hint plus one program-*restricting *check (the async-return "must be global `Promise`" rule) and never participates in* $[\![\cdot]\!]$*, subtyping, or refinement, so the model omits it.*

impl: `TType.sk / TField, TFields, TIndexesBase, FieldAccess`

Definition 12 (Signature Modes)**.** *The three-valued call/new encoding has the following intended reading (abbreviate:* None */* $\mathsf{Maybe}(\sigma)$ */* $\mathsf{Definite}(\sigma)$*):* None *= definitely not callable (constructible);* (true, σ) *=* maybe *callable, with signature* σ *if so;* (false, σ) *= definitely callable with signature* σ*. The implementation violates the* None *reading at class-constructor* values*: they are typed* None *although class constructors implement* [[Call]] *(`typeof C === "function"` at runtime), so* R-TYPEOF*'s soundness proof — which consumes this reading — inherits a witnessed violation at those values (ledger row A-CTOR-CALL; issue draft `class-constructor-call-none-typeof.md`; deep dive A). The same* None *mode is read as "callability unknown" by one subtyping arm (deep dive D's F-CALL) — the `call=None` dichotomy: no single reading validates both consumers.*

Definition 13 (Object-Type Semantics)**.** $o \in [\![\mathsf{object}(B)]\!]$ *iff* $o \in \mathsf{ObjVal}$ *and there exists an object branch* $b \in B$ *with* $o \in [\![b]\!]$*, where* $o \in [\![b]\!]$ *iff all of:*

- ***Required fields:*** *for every* $f \in \mathsf{flds}(b) \cup \mathsf{symflds}(b)$ *with* $\mathsf{opt}(f) = \mathsf{false}$*:* $\mathsf{has}(o, \mathsf{name}(f))$ *and, unless* $\mathsf{access}(f) = \mathsf{Phantom}$*,* $\mathsf{get}(o, \mathsf{name}(f)) \in [\![\mathsf{get}(f)]\!]$*.*
- ***Optional fields:*** *for every such* f *with* $\mathsf{opt}(f) = \mathsf{true}$*: if* $\mathsf{has}(o, \mathsf{name}(f))$ *then* $\mathsf{get}(o, \mathsf{name}(f)) \in [\![\mathsf{get}(f)]\!]$*.*
- ***Indexes:*** *an index constrains the props it* covers*: the string index covers every string key (of either* strKey *or* numKey *kind), the number index covers* numKey*-kind keys only, and the symbol index covers symbol keys — matching TS, where a string index signature also constrains numeric properties, and the implementation's numeric lookup (`numberIndex ?? stringIndex`). For each index* $\mathsf{idx}_\kappa(b) = \mathsf{Some}(f)$ *and every* $p \in \mathsf{props}(o)$ *covered by it and not named by a field of* b*:* $\mathsf{get}(o, p) \in [\![\mathsf{get}(f)]\!]$*.*
- ***Call/new:*** *if* $\mathsf{call}(b) = \mathsf{None}$ *then* $\neg\mathsf{callable}(o)$*; if* $\mathsf{call}(b) = (\mathsf{false}, \sigma)$ *then* $\mathsf{callable}(o)$*; if* $\mathsf{call}(b) = (\mathsf{true}, \sigma)$*, no constraint. Signatures themselves are opaque (their behavioral meaning is bundled into assumption* ***L-SUB****). Same for* new *with* constructible*.*
- ***Brand:*** *if* $\mathsf{isArr}(b) = \mathsf{Some}(x)$ *then* $\mathsf{isArrayV}(o) = x$*.*

The semantics is extensible*: properties not mentioned are unconstrained. A* Phantom *field constrains only* presence*, not the value — it records knowledge obtained from a runtime* `in` *check when no declaration is known; the implementation additionally makes phantoms error at access and be invisible to subtyping, which only* restricts *programs and so needs no semantic counterpart. The same restrict-only argument covers* Never *(produced when two same-named private fields with different owners meet: accessing it is a static error, but as a membership constraint it is an ordinary field). Note that membership constrains values via* get*-types only; the* set *component governs which writes the checker accepts and participates in soundness through subtyping and mutation, not through membership (assumption* ***L-MUT****). That governance has a second named facet, cross-view coherence: a checked write through one mutable view preserves the membership guarantees of every live alias of the same runtime object (*`L-MUT-ALIAS`*) — which holds only while every mutable view's* set*-types stay at or below the object's actual write capability. For recursive declarations the membership conditions recurse — a field's* $[\![\mathsf{get}(f)]\!]$ *can mention* $[\![b]\!]$ *again — and every recursive occurrence is positive, so least and greatest solutions exist (Knaster–Tarski);* $[\![\cdot]\!]$ *is the* greatest *solution, the reading under which a cyclic value (*`const x = {}; x.next = x`*) inhabits its recursive type. This choice is registry row* ***SEM-GFP****.*

impl: `TType.sk / TObject (interpretation comment), TFields`

Definition 14 (Write Authorization)**.** *A view b (an object branch, as recorded for a local or a field)* authorizes *the write of value v at key p iff* $v \in [\![\mathsf{setty}(b,p)]\!]$*, where* $\mathsf{setty}(b,p)$ *is the resolved* set*-type of p in b: the intersection* $\bigsqcap f.\mathsf{set}$ *of a named field f, else the covering index's* set*-type; a readonly field (*$\mathsf{set} = \bot$*) authorizes nothing. The checker realizes this definition at assignment sites — the right-hand side is checked against* setty *by subtyping (*`L-SUB`*) — and writes mediated by method calls are covered by the signature semantics (E-SIG), not by* setty*. This is the corpus's definition of a view's write surface, registry row* ***SET-SEM****;* `L-MUT-ALIAS` *is the coherence property over it.*

impl: `TType.sk / TFields::resolveFieldType (isLValue); TypeCheckExpr.sk / assign`

3.3 The union spine

Definition 15 (STypes)**.** STypes *is a finite, tag-indexed set: at most one* SType *per tag;* $[\![S]\!] = \bigcup_{s\in S}[\![s]\!]$*. Write* $S[k]$ *for the representative at tag k if present.*

impl: `TType.sk / STypes (InternalKeyedArrayMap keyed by STypeTag)`

Precision within a tag is carried by payloads: literal sets for primitives and object-branch sets for objects. A union of object types is represented exactly — one object branch per alternative — so merging alternatives loses no precision at the object tag.

Definition 16 (Instances)**.** TInst *is the abstract instance layer: type variables* α *(*`TVar`*, carrying a set of constraint types), unexpanded named instantiations (*`TSInst`*), unexpanded intersections (*`TInter`*), and delayed array-literal forms (*`TArrayLit`*).* TInsts *is a finite set of instances with* $[\![I]\!]_\rho = \bigcup_{i\in I}[\![i]\!]_\rho$*. Instance semantics is* parametrized by a substitution state ρ *(Section 6). The denotation of a type-variable occurrence* α^{cs} *(identifier* α *carrying the occurrence-level constraint set cs) is*

$$[\![\alpha^{cs}]\!]_\rho = \mathcal{E}_\rho(\alpha) \cap \bigcap_{c\in cs} [\![c]\!]_\rho, \qquad \mathcal{E}_\rho(\alpha) = \begin{cases} \emptyset & \alpha \notin \mathrm{dom}(\rho), \\ [\![t]\!]_\rho & \rho(\alpha) \in \{\mathsf{SVOpen}(t), \mathsf{SVClosed}(t)\}. \end{cases}$$

A binding type t may itself mention type variables, so this equation joins the recursive membership conditions of Definition 13; every recursive occurrence is positive, and the solution is the greatest

one (registry row SEM-GFP). An unbound `TVar` *thus denotes an empty inference lower bound, and* `TVar` *is the model's only variable kind — the model has no rigid or skolem variables. Constraints belong to occurrences because* `TVar::setConstraint` *is functional;* `Subst::get` *realizes the displayed intersection when the occurrence is read. This equation is the corpus's definition of* `TVar` *denotation, registry row* ***TVAR-SEM****. Named instances and intersections satisfy the expansion axiom* ***L-EXP****.*

impl: `TType.sk / TInst, TVar, TSInst, TArrayLit; TTypeInter.sk / TInter`

Definition 17 (Clause, Clauses, TType)**.**

A clause $\langle V \mid S \rangle$ *pairs type parameters* $V \subseteq_{\text{fin}} \mathsf{TParam}$ *with* $S \in \mathsf{STypes} \cup \{\top\}$*; clauses cannot be bottom (an empty* STypes *is dropped at construction).* Clauses *is a map from var-sets to clauses, at most one per key, with the invariant that if the top clause* $\langle \emptyset \mid \top \rangle$ *is present it is alone.* $\mathsf{TType} = (I, C)$ *with* $I \in \mathsf{TInsts}$*,* $C \in \mathsf{Clauses}$*.*

$$[\![\langle V \mid S \rangle]\!]_\rho = \bigcap_{v \in V} [\![v]\!]_\rho \cap [\![S]\!]_\rho \qquad [\![C]\!]_\rho = \bigcup_{c \in C} [\![c]\!]_\rho \qquad [\![(I, C)]\!]_\rho = [\![I]\!]_\rho \cup [\![C]\!]_\rho$$

where $[\![\top]\!]_\rho = \mathsf{Val}$*.*

A clause may have $V \neq \emptyset$ *with* $S = \top$ *(a type bounded only by its parameters); only* $\langle \emptyset \mid \top \rangle$ *is the top clause.*

impl: `TType.sk / Clause, Clauses, TType`

3.4 Operation interfaces

Refinement and constant removal are built from a small set of operations whose soundness this document assumes as assumptions and uses freely:

Axiom 18 (L-UNION, L-INTER)**.** *For all* t_1, t_2 *and states* ρ*:* $[\![t_1]\!]_\rho \cup [\![t_2]\!]_\rho \subseteq [\![\mathsf{union}(t_1, t_2)]\!]_\rho$ *and* $[\![t_1]\!]_\rho \cap [\![t_2]\!]_\rho \subseteq [\![\mathsf{inter}(t_1, t_2)]\!]_\rho$*. In particular* $\mathsf{inter}(t_1, t_2) = \bot$ *implies* $[\![t_1]\!]_\rho \cap [\![t_2]\!]_\rho = \emptyset$*, which refinement uses to drop branches. (The two clauses are tracked as separate registry rows: the containment as L-INTER, the disjointness clause as L-INTER-BOT.) The same containments are assumed at the* STypes *and* object *levels.*

impl: `TTypeUnion.sk; TTypeInter.sk; TType.sk / STypes::union, TObject::inter`

Axiom 19 (L-EXP: Expansion)**.** *Expansion rewrites a type to an instance-plus-clauses view using the current substitution state, possibly evolving that state (*$\rho \to \rho'$*, e.g. sealing a fresh covariantly-read variable to top). We assume* $[\![t]\!]_\rho \subseteq [\![\mathsf{expand}(t)]\!]_{\rho'}$ *and that residual instances (those left in the instance layer after expansion) keep their semantics. All refinement operators run on the expanded view. Expansion takes a mode (*`expandTVars`*); the containment is claimed for modes* `None/Covariant` *only — contravariant expansion is an under-approximating read belonging to* ***L-SUB*** *— and every refinement client uses those modes (deep dive C).*

impl: `TType.sk / TType::expand, expandToVectors, mapSTypes`

Axiom 20 (L-MONO: Refinement-Time State Monotonicity)**.** *The substitution evolutions performed* while refining a condition *(expansion reads, which may seal fresh variables to top, and exploratory intersections) preserve membership for the types* installed in the environment *at the evolution, and for the type the rule just produced: if* $\rho \to \rho'$ *is such an evolution, then for every* $u \in \mathrm{range}(\Gamma)$*,* $v \in [\![u]\!]_\rho \implies v \in [\![u]\!]_{\rho'}$*. Without this, narrowing one variable could silently invalidate the recorded types of* other *variables. Two remarks. First, deep dive E proves this axiom*

*outright (given semantic monotonicity in variable envelopes and **L-UNION**); in particular there is no constraint-mutation threat from shared state — `TVar::setConstraint` is a* functional *update on a frozen value, so constraints are occurrence-level (registry row SETCONSTRAINT-FUN) — and the* range(Γ) *restriction, stated here for safety, is droppable. Second, the classic contravariance counterexample (function parameter positions) is deflected only because signatures are opaque to $[\![\cdot]\!]$ — membership never inspects parameter types (Definition 13). This is the refinement-time counterpart of Obligation 47 (which concerns inference-time growth), and it is used by every branch-soundness argument below.*

impl: `TTypeSubst.sk / Subst::get (seal-to-top); TypeCheckRefine.sk (cloned-subst intersections)`

4 Refinement (Control-Flow Narrowing)

4.1 The single refinement primitive

Nearly all narrowing in SKJS is performed by one primitive applied to a local variable's type; the exceptions (equality's whole-variable intersection and the connectives' scope merge) are called out after the framework lemmas and receive direct proofs. The primitive is a filter/map: a partial mapper transforms each SType member of each clause of the expanded type, and an optional default replaces each *top* clause.

Definition 21 (Memberwise Refinement)**.** *Let $g : \mathsf{SType} \rightharpoonup \mathsf{SType}$ be a partial, tag-preserving mapper and $d \in \{\mathsf{None}\} \cup \{\mathsf{Some}(D) \mid D \in \mathsf{STypes}\}$ an optional default. Define $\mathsf{refine}^{g,d}(t)$ as follows: expand t (axiom **L-EXP**) to residual instances I and clauses C; replace each clause $\langle V \mid S\rangle$ by*

$$\begin{cases} \langle V \mid D\rangle & \text{if } S = \top \text{ and } d = \mathsf{Some}(D) \\ \langle V \mid \top\rangle & \text{if } S = \top \text{ and } d = \mathsf{None} \\ \langle V \mid \{g(s) \mid s \in S,\ g(s) \text{ defined}\}\rangle & \text{otherwise,} \end{cases}$$

dropping clauses whose STypes *becomes empty; the result is (I, C') rebuilt with the clause union.*

impl: `TypeCheckRefine.sk / setLocalFilterMap; TType.sk / TType::mapSTypes`

Definition 22 (Predicate Soundness)**.** *Let $P \in \mathcal{P}(\mathsf{Val})$. A partial mapper g is* sound for P *if for every $s \in \mathsf{SType}$ and $v \in [\![s]\!] \cap P$: $g(s)$ is defined and $v \in [\![g(s)]\!]$. A default $d = \mathsf{Some}(D)$ is* adequate for P *if $P \subseteq [\![D]\!]$; $d = \mathsf{None}$ is always adequate. Object-typed members make these denotations state-indexed (their field types may mention instances); throughout, $[\![s]\!], [\![g(s)]\!], [\![D]\!]$ are read at the post-refinement state ρ'. A transformer $R : \mathsf{TType} \to \mathsf{TType}$ is* predicate-sound for P *if $[\![t]\!]_\rho \cap P \subseteq [\![R(t)]\!]_{\rho'}$ for all t (with $\rho \to \rho'$ the state evolution of R).*

Lemma 23 (Memberwise Refinement is Predicate-Sound)**.** *If g is sound for P and d adequate for P, then $\mathsf{refine}^{g,d}$ is predicate-sound for P.*

Proof. Let $v \in [\![t]\!]_\rho \cap P$. By **L-EXP**, $v \in [\![(I, C)]\!]_{\rho'}$ for the expanded view. If $v \in [\![I]\!]$, the instance layer is unchanged and v survives. Otherwise $v \in [\![\langle V \mid S\rangle]\!]$ for some clause. If $S = \top$: with $d = \mathsf{None}$ the clause is unchanged; with $d = \mathsf{Some}(D)$, $v \in \bigcap_V [\![\cdot]\!]$ still, and $v \in P \subseteq [\![D]\!]$, so $v \in [\![\langle V \mid D\rangle]\!]$. If $S \neq \top$: $v \in [\![s]\!]$ for some $s \in S$, and by soundness of g, $g(s)$ is defined with $v \in [\![g(s)]\!]$, so the clause (which is then nonempty) retains v; clauses dropped for emptiness never contained v. Tag preservation of g is what makes the image a well-formed STypes (at most one member per tag), so no two survivors merge or clobber. Rebuilding with clause union only enlarges (**L-UNION**). □

Lemma 24 (Identity is Sound)**.** *The identity transformer is predicate-sound for every P.* □

Beyond the primitive. Two rule families do not factor through $\mathsf{refine}^{g,d}$: equality narrowing on a whole variable, which *replaces* the type by an intersection (Rule R-Eq); and the connective rules, which merge two refined scopes (Rule R-And/R-Or). They get direct proofs.

4.2 Environments, evaluation, and branch soundness

Definition 25 (Environments and Stores)**.** Γ *maps the* local *variables of the current scope to* TType*;* σ *maps locals to* Val*. Write* $\sigma \models_\rho \Gamma$ *when* $\sigma(x) \in [\![\Gamma(x)]\!]_\rho$ *for all* $x \in \mathrm{dom}(\Gamma)$*. Only locals are narrowed: the implementation's refinement primitive is a no-op for bindings not in the local scope (module-level declarations), since narrowing a global is unsound under concurrent mutation through closures or aliases. (Assumption* ***L-LOCAL****.)*

impl: `TypeCheckRefine.sk / setLocalFilterMap (non-local early return)`

Definition 26 (Evaluation of Condition Forms)**.** *For each condition form* e *of the master list we define relations* $\sigma \models_{\mathsf{true}} e$ *and* $\sigma \models_{\mathsf{false}} e$*, capturing the runtime outcomes under which the corresponding branch is taken. Evaluation of subexpressions is assumed not to modify* σ *(assumption* ***L-PURE****); forms whose evaluation would throw (e.g.* `"p" in 3`*) satisfy neither relation, which makes any narrowing of an unreachable branch vacuously sound.*

- $\sigma \models_b (e_1 \texttt{ === } e_2)$ *iff* $v_1 \approx v_2$ *(Definition 4) exactly when* $b = \mathsf{true}$*, where* v_1, v_2 *are the values of* e_1, e_2*.* `!==` *swaps* b*.*
- $\sigma \models_b (\texttt{typeof } x \texttt{ === } k)$ *iff* $\sigma(x) \in P_{\mathsf{typeof}=k}$ *exactly when* $b = \mathsf{true}$*, with*

$$P_{\mathsf{typeof}=\texttt{"undefined"}} = \{v \mid \mathsf{rtag}(v) \in \{\mathsf{undef}, \mathsf{void}\}\}$$
$$P_{\mathsf{typeof}=\texttt{"object"}} = [\![\texttt{null}]\!] \cup \{o \in \mathsf{ObjVal} \mid \neg\mathsf{callable}(o)\}$$
$$P_{\mathsf{typeof}=\texttt{"function"}} = \{o \in \mathsf{ObjVal} \mid \mathsf{callable}(o)\}$$

  and $P_{\mathsf{typeof}=k}$ *the evident tag set for* $k \in \{\texttt{"boolean"}, \texttt{"number"}, \texttt{"string"}, \texttt{"bigint"}, \texttt{"symbol"}\}$*.*

- $\sigma \models_{\mathsf{true}} (\texttt{"p" in } x)$ *iff* $\sigma(x) \in \mathsf{ObjVal}$ *and* $\mathsf{has}(\sigma(x), p)$*;* $\models_{\mathsf{false}}$ *iff* $\sigma(x) \in \mathsf{ObjVal}$ *and* $\neg\mathsf{has}(\sigma(x), p)$*. (A non-object* $\sigma(x)$ *throws.)*
- $\sigma \models_{\mathsf{true}} (x \texttt{ instanceof } C)$ *iff* $\sigma(x) \in P_{\mathsf{inst}}(C) \subseteq \mathsf{ObjVal}$ *(assumption* ***L-INST****).*
- $\sigma \models_{\mathsf{true}} \texttt{Array.isArray}(x)$ *iff* $\sigma(x) \in \mathsf{ObjVal}$ *and* $\mathsf{isArrayV}(\sigma(x))$*;* $\models_{\mathsf{false}}$ *is the complement over all of* Val*.*
- $\sigma \models_b (\texttt{!}e)$ *iff* $\sigma \models_{\neg b} e$*.*
- $\sigma \models_{\mathsf{true}} (e_1 \texttt{ \&\& } e_2)$ *iff* $\sigma \models_{\mathsf{true}} e_1$ *and* $\sigma \models_{\mathsf{true}} e_2$*;* $\sigma \models_{\mathsf{false}} (e_1 \texttt{ \&\& } e_2)$ *iff* $\sigma \models_{\mathsf{false}} e_1$*, or* $\sigma \models_{\mathsf{true}} e_1$ *and* $\sigma \models_{\mathsf{false}} e_2$*. Dually for* `||`*.*
- $\sigma \models_b x$ *(truthiness of a variable) iff* $\mathsf{truthy}(\sigma(x)) = b$*; likewise* $\sigma \models_b x.f$ *iff* $\sigma(x) \in \mathsf{ObjVal}$*,* $\mathsf{has}(\sigma(x), f)$ *or the read yields* `undefined`*, and the read value's truthiness is* b*.*

Definition 27 (Branch Soundness)**.** *A rule deriving* $\Gamma \vdash e \overset{b}{\rightsquigarrow} \Gamma'$ *is* branch-sound *if* $\sigma \models_\rho \Gamma$ *and* $\sigma \models_b e$ *imply* $\sigma \models_{\rho'} \Gamma'$*, where* $\rho \to \rho'$ *is the substitution evolution performed by the rule's refinement operators (an evolution of the kind covered by* ***L-MONO****).*

Lemma 28 (Predicate-Sound Update is Branch-Sound)**.** *If* R *is predicate-sound for* P*, the update* $\Gamma' = \Gamma[x \mapsto R(\Gamma(x))]$ *is branch-sound for any* e, b *with* $\sigma \models_b e \implies \sigma(x) \in P$*.*

Proof. For unchanged variables y: $\sigma(y) \in [\![\Gamma(y)]\!]_\rho$, and **L-MONO** carries this to ρ' — this step is *not* free, as the semantics of $\Gamma(y)$ is state-indexed. For x: $\sigma(x) \in [\![\Gamma(x)]\!]_\rho \cap P \subseteq [\![R(\Gamma(x))]\!]_{\rho'}$ by predicate-soundness of R. □

4.3 Master list

Each row corresponds to *one dispatch arm* of `evalCond` (`TypeCheckRefine.sk`); the maintenance check of Appendix B keeps this correspondence mechanical. "id" means that control-flow branch performs no narrowing.

Rule	Condition form	Branches	Impl (dispatch arm / handler)
R-Eq	x `===` e, x `===` y	both	`evalEquality`/`evalVarEquality`
R-EqUndef	x `===` `undefined`	both	`evalVarEquality` (undefined case)
R-EqField	$x.f$ `===` e (and $x.f$ `===` $y.g$)	both	`evalFieldEqualityType`, `narrowField`
R-Typeof	`typeof` x `===` k	both	`evalTypeofEquality`
R-IsArray	`Array.isArray(`x`)`	both	`evalIsArray`
R-Inst	x `instanceof` C	true only	`evalInstanceof`
R-In	e_k `in` x	both	`evalCond` (`In` arm)
R-Not	`!`e	both	`evalCond` (`Unop` arm)
R-And	e_1 `&&` e_2	both	`evalCond` (`BOLogicalAnd` arm)
R-Or	e_1 `||` e_2	both	`evalCond` (`BOLogicalOr` arm)
R-Truthy	x in condition position	both	`evalTruthiness`
R-TruthyField	$x.f$ in condition position	both	`evalFieldTruthiness`
R-Other	anything else	both (id)	`evalCond` final wildcard

Entry points that reduce to this list: `if`/`while` conditions, the ternary c `?` a `:` b (which narrows inside a and b exactly like `if`), and `&&`/`||` value positions call `evalCond`; `switch` desugars each `case` v`:` to e `===` v (`TypeCheckStmt.sk`); `console.assert`(e) applies the true branch of e — matched by call shape, first argument, without verifying `console` is the global (`TypeCheckExpr.sk`). `for`-loop conditions do *not* narrow. Strict inequality dispatches to the same handlers with the target negated.

4.4 Rules and soundness

Throughout, b is the branch (true in the taken/then branch, false in the untaken/else branch) and updates go through Lemma 28 unless noted.

4.4.1 R-Eq: strict equality on a variable

For x `===` e with e not the literal `undefined` (that case is R-EqUndef), let τ_e be the checked type of e.

$$\frac{}{\Gamma \vdash (x \texttt{ === } e) \overset{\mathsf{t}}{\rightsquigarrow} \Gamma[x \mapsto \mathsf{inter}(\Gamma(x), \tau_e)]}\ \text{R-EqT} \qquad \frac{\mathsf{singleConst}(\tau_e) = \mathsf{Some}(c)}{\Gamma \vdash (x \texttt{ === } e) \overset{\mathsf{f}}{\rightsquigarrow} \Gamma[x \mapsto \mathsf{rmc}(\Gamma(x), c)]}\ \text{R-EqF}$$

where rmc is constant removal (Section 5) and $\mathsf{singleConst}$ succeeds only when τ_e is a single unit or single-literal primitive, whose denotation is then *contained in* a single $\approx$-class — equal to it for

literal primitives and the `null` unit, a proper subset for the `undefined`/`void` units, whose cross-tag partner Definition 8 deliberately excludes. If $\mathsf{singleConst}$ fails, the false branch is the identity. When both sides are variables (or a variable and a field path), the rule applies to each side independently.
impl: `TypeCheckRefine.sk / evalVarEquality`

Lemma 29 (R-Eq is Branch-Sound)**.** *Assume $[\![\tau_e]\!]$ is $\approx$-closed (assumption **L-SEQ**).*

Proof. True branch. $\sigma \models_{\mathsf{true}}$ gives $\sigma(x) \approx v_e$, the value of e. By type correctness of the checker for e (assumption **L-TC**), $v_e \in [\![\tau_e]\!]$, and by $\approx$-closure $\sigma(x) \in [\![\tau_e]\!]$. So $\sigma(x) \in [\![\Gamma(x)]\!] \cap [\![\tau_e]\!] \subseteq [\![\mathsf{inter}(\Gamma(x), \tau_e)]\!]$ by **L-INTER**. As in Lemma 28, the unchanged variables transport to the post-refinement state by **L-MONO** (this rule bypasses the memberwise primitive, so the step must be taken explicitly). *False branch.* $\sigma(x) \not\approx v_e$, and every member of $[\![c]\!]$ is $\approx v_e$ (Lemma 5 for a literal constant, since $v_e \in [\![\tau_e]\!] = [\![c]\!]$; unit reflexivity otherwise), so $\sigma(x) \notin [\![c]\!]$; then $\sigma(x) \in [\![\Gamma(x)]\!] \setminus [\![c]\!] \subseteq [\![\mathsf{rmc}(\Gamma(x), c)]\!]$ by Theorem 40(R2). □

The $\approx$-closure hypothesis. It holds by construction for literal and full primitive types (Definition 8 takes the same-tag closure, using transitivity via Lemma 5, and $\approx$ is payload equality on non-numeric tags), for the `null` unit (payload equality at its own tag — null comparisons flow through this generic rule), and trivially for object types (reference equality). The corner where it can fail is the cross-tag $\mathsf{undef}/\mathsf{void}$ arm of $\approx$, in either direction: a τ_e denoting the `undefined` unit alone is not closed against void-tagged values, and dually for a `void`-typed RHS meeting an undefined-tagged value. The syntactic comparison x `===` `undefined` takes the dedicated rule R-EqUndef, which treats the two tags jointly; a comparison against an undefined- or void-*typed* expression that is not the literal (`x === y` with `y: undefined`) reaches the generic rule — but the corner where either unit type could meet the other tag's value is unreachable: the checker rejects `void`-typed operands of strict equality outright ("An expression of type 'void' cannot be used here"), before any refinement runs, discharging the last case of assumption **L-SEQ** (empirically pinned).

State caveat. The implementation computes the intersection under a *cloned* substitution and installs the resulting type against the main one; assumption **L-CLONE** records the stability requirement this creates.

4.4.2 R-EqUndef: comparison with `undefined`

Mapper g_b: keep s iff $(\mathsf{tag}(s) \in \{\mathsf{undef}, \mathsf{void}\}) = b$; no default. This is $\mathsf{refine}^{g_b, \mathsf{None}}$ on both branches. Soundness: $\sigma \models_b$ holds iff $(\sigma(x) \approx (\mathsf{undef}, \star)) = b$, and by Definition 4 that is exactly $(\mathsf{rtag}(\sigma(x)) \in \{\mathsf{undef}, \mathsf{void}\}) = b$ — the rule treats the two tags jointly precisely because $\approx$ identifies them. Apply Lemmas 23 and 28. (`null` comparisons are *not* special-cased here: they flow through R-Eq with $c =$ `null`.)
impl: `TypeCheckRefine.sk / evalVarEquality (undefined case)`

4.4.3 R-EqField: strict equality on a field path

For $x.f$ `===` e with RHS type τ_e and $c = \mathsf{singleConst}(\tau_e)$ (possibly undefined), the mapper acts only on object branches. For each branch β of each object member of $\Gamma(x)$, let τ_β be β's *resolved read type* for the *declared named* field f: $\mathsf{get}(f)$ if f is required, $\mathsf{union}(\mathsf{get}(f), \mathtt{undefined})$ if optional (`maybeGetType`/`resolveFieldType` — the implementation resolves before narrowing, which is load-bearing below). If β has no declared field f — including when f is reachable only through an index

signature — the object branch is kept unchanged (a sound over-approximation; no diagnostic is emitted). Let $\omega = \mathsf{inter}(\tau_\beta, \tau_e)$:

$$g_{\mathsf{true}}(\beta) = \begin{cases} \text{drop} & \omega = \bot \\ \beta[f.\mathsf{get} \mapsto \omega] & \text{else} \end{cases} \qquad g_{\mathsf{false}}(\beta) = \begin{cases} \beta & \omega = \bot \text{ or } c \text{ undefined} \\ \text{drop} & \mathsf{rmc}(\tau_\beta, c) = \bot \\ \beta[f.\mathsf{get} \mapsto \mathsf{rmc}(\tau_\beta, c)] & \text{else} \end{cases}$$

with the bookkeeping that narrowing out `undefined` also clears the field's optionality.[1] Non-object members are kept unchanged on both branches. The narrowed read type is written back as the field's get; when the field stays optional, the next read re-adds `undefined`, which is conservative.
impl: `TypeCheckRefine.sk / evalFieldEqualityType, narrowField`

Lemma 30 (R-EqField is Branch-Sound)**.** *Assume $[\![\tau_e]\!]$ is $\approx$-closed (**L-SEQ**), as in* R-Eq.

Proof. Let $v = \sigma(x) \in [\![\Gamma(x)]\!]$ and let the read value be $w = \mathsf{get}(v, f)$ if present, else the value `undefined`. For an object branch β with declared field f, membership of v in β gives $w \in [\![\tau_\beta]\!]$ *by construction of the resolved read type*: if f is present in v, Definition 13 gives $w \in [\![\mathsf{get}(f)]\!] \subseteq [\![\tau_\beta]\!]$ (**L-UNION**); if f is optional and absent, $w =$ `undefined` $\in [\![\tau_\beta]\!]$ because resolution unioned `undefined` in. (This is why resolving before narrowing is load-bearing: with the raw get-type, the true branch of `x.f === undefined` on an absent optional field would compute $\omega = \bot$ and wrongly drop the object branch.) *True branch:* $w \approx v_e$ and $v_e \in [\![\tau_e]\!]$ (**L-TC**), so $w \in [\![\tau_e]\!]$ by closure, and $w \in [\![\omega]\!]$ by **L-INTER**; in particular $\omega \neq \bot$, so v's object branch is not dropped. If f is present in v, the strengthened field is satisfied; if absent (possible only when $v_e \approx$ `undefined`), optionality is kept and the strengthened field imposes no constraint. *False branch:* $w \not\approx v_e$. If c is defined then every member of $[\![c]\!]$ is $\approx v_e$ (as in R-Eq), so $w \in [\![\tau_\beta]\!] \setminus [\![c]\!] \subseteq [\![\mathsf{rmc}(\tau_\beta, c)]\!]$ by Theorem 40(R2); since w inhabits it, the removal is not $\bot$ and the object branch survives with v in it. When c is the `undefined` unit, optionality is cleared: $w \not\approx$ `undefined` entails f is present in v, so the now-required field is satisfied. If $\omega = \bot$ then by **L-INTER** the field value could never strictly equal v_e (using closure of $[\![\tau_e]\!]$), and the object branch is kept unchanged, which is sound. Branches without a declared f and non-object members are unchanged (identity, Lemma 24). □

4.4.4 R-Typeof

Eight recognized operands; any other string is a static error (TS2367), and then no narrowing occurs. Writing b for the control-flow branch:

- $k \in \{$`"boolean"`, `"number"`, `"string"`, `"bigint"`, `"symbol"`$\}$: keep s iff $(\mathsf{tag}(s) = \mathsf{tag}(k)) = b$; default on the true branch only: the full primitive of kind k.
- $k =$ `"undefined"`: keep s iff $(\mathsf{tag}(s) \in \{\mathsf{undef}, \mathsf{void}\}) = b$; no default.
- $k =$ `"object"`: on $b = \mathsf{true}$, keep `null`; drop other primitives; per object branch, by call mode (Definition 12): $\mathsf{None} \to$ keep, $(\mathsf{true}, \sigma) \to$ keep with call mode None, $(\mathsf{false}, \sigma) \to$ drop. On $b = \mathsf{false}$: dual — `null` dropped, other primitives kept, per object branch $\mathsf{None} \to$ drop, $(\mathsf{true}, \sigma) \to (\mathsf{false}, \sigma)$, $(\mathsf{false}, \sigma) \to$ keep. Default (true branch): $\{$`null`$\} \sqcup \mathsf{object}(\{\emptyset\})$.
- $k =$ `"function"`: on $b = \mathsf{true}$: drop all non-objects; per object branch $\mathsf{None} \to$ drop, $(\mathsf{true}, \sigma) \to (\mathsf{false}, \sigma)$, $(\mathsf{false}, \sigma) \to$ keep. On $b = \mathsf{false}$: keep non-objects; per object branch $\mathsf{None} \to$ keep, $(\mathsf{true}, \sigma) \to$ call mode None, $(\mathsf{false}, \sigma) \to$ drop. No default.

[1] The optionality-clearing test matches only the `undefined` unit, not `void`; subtracting a `void`-typed constant keeps optionality — conservative, hence sound.

impl: `TypeCheckRefine.sk / evalTypeofEquality`

Lemma 31 (R-TYPEOF is Branch-Sound)**.**

Proof. Tag cases are immediate from A0 and Definition 26; defaults are adequate since $P_{\mathsf{typeof}=k} \subseteq [\![\text{full } k]\!]$. For `"object"`, the default's object branch (`TObject::createEmpty`) has no fields or indexes and call = new = None, which by Definition 13 constrains members to be non-callable and non-constructible; every $o \in P_{\mathsf{typeof}=\texttt{"object"}} \setminus [\![\texttt{null}]\!]$ is non-callable by definition and therefore non-constructible by the constructible $\implies$ callable fact of Definition 3, so adequacy holds. For the callable cases, let $v \in \mathsf{ObjVal} \cap [\![\beta]\!]$. If $\mathsf{callable}(v)$ ($k =$ `"function"`, $b = \mathsf{true}$): β's call mode cannot be None (Definition 13); a (true, σ) mode is strengthened to (false, σ), which v satisfies since it is callable. If $\neg\mathsf{callable}(v)$: the mode cannot be (false, σ); a (true, σ) mode weakens/strengthens to None, which v satisfies. The `"object"` cases combine this with the `null` tag case. The mode rewrites preserve all other constraints of β, and are tag-preserving, so Lemma 23 applies. □

4.4.5 R-IsArray

True branch: drop non-object members; each object branch is intersected with the single object branch of object(ReadonlyArray$\langle\top\rangle$) and its brand set to Some(true); an object branch whose intersection is $\bot$, or that was already branded Some(false), is dropped. (The `ReadonlyArray` stub itself carries the brand as a declared field `readonly __SKJS_isArray: true`, so the brand-conflict drop already occurs inside the intersection.) False branch: object branches have the brand set to Some(false); a branch already branded Some(true) is dropped; non-objects are kept.

impl: `TypeCheckRefine.sk / evalIsArray; TTypeInter.sk / TFields::interIsArray`

Lemma 32 (R-ISARRAY is Branch-Sound)**.**

Proof. True branch: $\sigma \models_{\mathsf{true}}$ gives $v = \sigma(x) \in \mathsf{ObjVal}$ with $\mathsf{isArrayV}(v)$. By assumption **L-ISARR**, $v \in [\![\mathsf{object}(\mathsf{ReadonlyArray}\langle\top\rangle)]\!]$; by **L-INTER** (object level) v inhabits the intersected branch, and setting the brand to Some(true) is satisfied by $\mathsf{isArrayV}(v)$. False branch: if $v \in \mathsf{ObjVal}$ then $\neg\mathsf{isArrayV}(v)$ satisfies brand Some(false) and contradicts membership in a Some(true)-branded branch, so dropping those loses nothing; non-objects pass through. □

Mutability note. Intersecting with *Readonly*Array rather than `Array` is a deliberate precision/soundness choice documented in the implementation: a readonly-array-typed value may be a non-array object carrying the right properties in general, but under isArrayV the brand is known; what is *not* known is mutability, so the mutable interface is added only where the original branch already had it.

4.4.6 R-Inst: `instanceof`

Fires only on the true branch and only when the RHS names a class or interface with a known definition (a Function-typed value as RHS narrows nothing). Let $T_C =$ object-typed instance of C with implementation-chosen type arguments ($\top$ for `Array`; fresh inference arguments otherwise). Mapper: drop non-objects; intersect each object member with T_C (dropping empty intersections). Default: T_C itself.

impl: `TypeCheckRefine.sk / evalInstanceof, evalCond (InstanceOf arm)`

Lemma 33 (R-INST is Branch-Sound)**.** *Given assumption **L-INST** ($P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$, with T_C as instantiated at the narrowing site), the true-branch rule is branch-sound; the false branch is the identity.*

Proof. Let $v = \sigma(x)$ with $\sigma \models_{\mathsf{true}}$, so $v \in P_{\mathsf{inst}}(C) \subseteq \mathsf{ObjVal}$. Dropping non-object members is sound by A0 (values of a non-object member carry that member's tag, so v is not in it). For v's object member, $v \in [\![T_C]\!]$ by **L-INST**, so v inhabits the intersection by **L-INTER** (object level), which is in particular not $\bot$. The default $D = \{T_C\}$ is a singleton STypes (T_C is a `TObject` built by `newObject`) and is adequate by **L-INST** directly. Both **L-INST** and the mapper are evaluated at the post-instantiation state ρ' (T_C's fresh inference arguments live in ρ'); later growth of those arguments is covered by **L-MONO**/Obligation 47. Apply Lemmas 23 and 28. □

4.4.7 R-In

The key expression's *type* determines the refinement: if it denotes a single string *or numeric* literal p or a single (unique) symbol q, narrowing proceeds with key $\pi \in \{p, q\}$ — a numeric literal is converted to its string spelling and the key kind is hardcoded to string, so the true branch's index lookup below consults the *string* index even for a numeric key — sound under the covering semantics of Definition 13, since a string index covers numeric-kind keys too; the imprecision is only that a number index is not consulted. If the key type is a multi-literal or non-literal string/number/symbol type, object members are kept unchanged — but primitive members are still dropped on the true branch; if it is not a string/number/symbol type at all, the checker raises a *fatal* TS2360 and no narrowing (or further checking of the definition) occurs. In every non-fatal case the true branch drops primitive members (evaluation of `in` on a primitive throws; Definition 26). With a single key π, per object branch β:

- **True branch:** if β has field π: clear its optionality. Else if β has a non-synthetic index *covering* π with field f_{idx}: add the required field $\pi \mapsto f_{\text{idx}}$. Else: add a required *phantom* field π ($\mathsf{get} = \top$, $\mathsf{set} = \bot$, $\mathsf{access} = \mathsf{Phantom}$).
- **False branch:** if β has required π: drop the object branch. If optional π: remove the field from β. Else: keep.

There is no default: a top-clause type stays top on both control-flow branches.
impl: `TypeCheckRefine.sk / evalCond (In arm), inLhsFieldsFromType`

Lemma 34 (R-In is Branch-Sound)**.**

Proof. True branch: $v = \sigma(x) \in \mathsf{ObjVal}$ with $\mathsf{has}(v, \pi)$. Primitives cannot contain v (A0), so dropping them is sound. If β had optional π, presence upgrades v's satisfaction to the required form; the value constraint is unchanged. If the field is added from an index covering π: membership of v in β already constrains $\mathsf{get}(v, \pi) \in [\![f_{\text{idx}}.\mathsf{get}]\!]$ (Definition 13, index clause — π is covered and was not named), so v satisfies the named copy. A phantom field constrains presence only, which holds. False branch: $v \in \mathsf{ObjVal}$ with $\neg\mathsf{has}(v, \pi)$ contradicts a required π — including a required *phantom*, which still demands presence — so v was never in dropped object branches. Removing an optional field π is *not* monotone in general (the removal re-exposes π to the object branch's index constraint, which the named field had exempted), but it is sound here: v lacks π, so the re-exposed index constraint on π is vacuous for v, and constraints on other keys are unchanged. Primitives kept is identity. □

4.4.8 R-Not, R-And, R-Or

$$\frac{\Gamma \vdash e \overset{\neg b}{\rightsquigarrow} \Gamma'}{\Gamma \vdash \,!e \overset{b}{\rightsquigarrow} \Gamma'}\ \text{R-NOT} \qquad \frac{\Gamma \vdash e_1 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_1 \qquad \Gamma_1 \vdash e_2 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2}{\Gamma \vdash (e_1 \,\&\&\, e_2) \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2}\ \text{R-ANDT}$$

$$\frac{\Gamma \vdash e_1 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_1 \qquad \Gamma \vdash e_1 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma' \qquad \Gamma' \vdash e_2 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_2}{\Gamma \vdash (e_1 \,\&\&\, e_2) \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_1 \sqcup \Gamma_2}\ \text{R-ANDF}$$

and dually R-ORF (sequential) and R-ORT (merge of "e_1 true" and "e_1 false then e_2 true"). Here $(\Gamma_1 \sqcup \Gamma_2)(x) = \mathsf{union}(\Gamma_1(x), \Gamma_2(x))$ is the pointwise scope merge. (The implementation merges only the bindings the sub-scopes updated: a variable narrowed in just one sub-scope is unioned with the *parent* scope's type — which equals the pointwise union, since the un-narrowed side's value is the parent's.)

impl: `TypeCheckRefine.sk / evalCond (Unop / BOLogicalAnd / BOLogicalOr arms); TypeCheckScope.sk / Scope::merge via TypeCheckEnv.sk / mergeScope`

Lemma 35 (Merge Soundness)**.** *If $\sigma \models \Gamma_1$ or $\sigma \models \Gamma_2$, then $\sigma \models \Gamma_1 \sqcup \Gamma_2$.*

Proof. Pointwise from **L-UNION**. The domains agree ($\mathrm{dom}(\Gamma_1) = \mathrm{dom}(\Gamma_2)$) because every rule is an update $\Gamma[x \mapsto \cdot]$ of an existing local — refinement never adds or removes bindings. The merged environment, like all post-refinement environments, is interpreted at the final state; the premises' satisfaction transports to it by **L-MONO**. □

Lemma 36 (Connectives are Branch-Sound)**.** *If all premise derivations are branch-sound, so are* R-NOT, R-AND, R-OR *— for* both *branches.*

Proof. R-NOT is immediate from Definition 26. R-ANDT composes directly: both conjuncts hold and the two refinements chain. For R-ANDF: by Definition 26, $\sigma \models_{\mathsf{false}} e_1$ — giving $\sigma \models \Gamma_1$ — or $\sigma \models_{\mathsf{true}} e_1$ and $\sigma \models_{\mathsf{false}} e_2$ — giving $\sigma \models \Gamma'$ then $\sigma \models \Gamma_2$. Either way Lemma 35 concludes. The `||` cases are dual. Note the exactness of the short-circuit semantics in Definition 26 is what licenses refining *both* control-flow branches of both connectives. □

4.4.9 R-Truthy, R-TruthyField

Definition 37 (Truthiness Residue)**.** *For $b \in \{\mathsf{true}, \mathsf{false}\}$, the partial mapper $\mathsf{tres}_b : \mathsf{SType} \rightharpoonup \mathsf{SType}$ is:*

- $\mathsf{unit}(\texttt{void})$*: kept unchanged on both branches (*`void`* is treated as non-observable and never narrowed);*
- $\mathsf{unit}(\texttt{undefined}), \mathsf{unit}(\texttt{null})$*: dropped if b, kept if $\neg b$;*
- *objects and symbols: kept if b, dropped if $\neg b$;*
- *full* `boolean`*: the literal b;*
- *full* `string`*/*`number`*/*`bigint`*: kept unchanged on* both *branches;*
- *literal primitives $\mathsf{prim}(k, L)$: keep the literals $\ell \in L$ with $\mathsf{truthy}(\delta_k(\ell)) = b$; undefined if none remain.*

The value *residue* vres_b *differs in exactly two cases: full* `string` *and full* `bigint` *on* $b = \mathsf{false}$ *narrow to the literals* `""` *and* `0n` *(their unique falsy values); full* `number` *must stay wide because* NaN *has no literal type.*

impl: TType.sk / SType::truthinessResidue, truthinessValueResidue

Lemma 38 (Residues are Sound)**.** tres_b *is sound (Definition 22) for* $P_b = \{v \mid \mathsf{truthy}(v) = b\} \cup [\![\texttt{void}]\!]$*, and* vres_b *is sound for* P_b *as well.*

Proof. Case analysis over Definition 6. The two deliberate widenings (full string/number/bigint kept on both branches) only enlarge the result. The literal-filtering case is sound *only if* the implementation's lexeme-level falsiness test agrees with Definition 6 on every literal in L — this is the falsiness-exactness part (iii) of assumption **L-LIT**, which the string kind violates (see the assumptions table for the full inventory of violated assumptions): a line-continuation string lexeme denotes `""` yet is classified truthy, and would be dropped from the falsy branch. For $\mathsf{vres}_{\mathsf{false}}$: the only falsy string value is `""` and the only falsy bigint value is 0, so the narrowed literal contains every falsy member; number stays wide precisely because its falsy set $\{0, -0, \mathsf{NaN}\}$ is not expressible as literals. The $[\![\texttt{void}]\!]$ disjunct in P_b accounts for keeping `void` on both branches (for $b = \mathsf{false}$ it is redundant, since void values are falsy). □

R-Truthy is $\mathsf{refine}^{\mathsf{tres}_b,\mathsf{None}}$ on $\Gamma(x)$; branch soundness follows from Lemmas 23, 38 and 28 (note $\sigma \models_b x$ implies $\sigma(x) \in P_b$, and `void`-typed reads evaluate to `undefined`, which the kept `void` member covers). R-TruthyField applies the *type-level* residue ($\mathsf{refine}^{\mathsf{tres}_b,\mathsf{None}}$ on the field's type) through the same per-branch field update as R-EqField, dropping object branches whose residue is $\bot$ and clearing optionality on the true branch (the read value being truthy entails the field is present with a non-`undefined` value); its proof combines Lemma 30's structure with Lemma 38.

impl: `TypeCheckRefine.sk / evalTruthiness, evalFieldTruthiness; TType.sk / TType::truthinessPart`

Value positions of `&&`/`||`. The *type of the expression* e_1 `&&` e_2 uses $\mathsf{vres}_{\mathsf{false}}$ on e_1's type for the untaken-operand contribution (and dually for `||`); this is a typing rule rather than a refinement and inherits soundness from Lemma 38.

impl: `TType.sk / TType::truthinessValuePart`

5 Constant Removal

SKJS has no general type-difference operation: inference growth is structural (Section 6) and consumes no differences, and the only subtraction clients — destructuring defaults ($\tau \setminus \texttt{undefined}$) and `!==` narrowing — subtract a *single constant*. (The historical `TType::diff` and its per-level strata were deleted when growth became structural.)

Definition 39 (Constant Removal)**.** *A* constant c *is a unit or a single-literal primitive; its denotation* $[\![c]\!]$ *is* contained in *a single* $\approx$*-class — equal to it for literal primitives (for a numeric constant this may be the two-element class* $\{0, -0\}$*) and the* `null` *unit; a proper subset for the* `undefined/void` *units, whose cross-tag partner Definition 8 deliberately excludes.* $\mathsf{rmc}(t, c)$ *expands* t *and maps each clause's* STypes *elementwise:*

- $\mathsf{unit}(u)$ *with* $\mathsf{tag}(u) = \mathsf{tag}(c)$*: dropped;*
- $\mathsf{prim}(k, L)$ *with* $\mathsf{tag}(k) = \mathsf{tag}(c)$ *and* $L \neq \top$*: becomes* $\mathsf{prim}(k, L \setminus \{\ell_c\})$*, dropped if empty;*
- *full* `boolean` *minus a boolean literal: the complementary literal (exact);*

- *full* `number/string/bigint/symbol`*: kept unchanged (over-approximation);*
- *all other members (different tag, objects): kept unchanged.*

Top clauses are unchanged. The instance layer is unchanged.
impl: `TType.sk / TType::removeConstant; PrimLiterals.sk / diff`

Theorem 40 (Soundness of Constant Removal)**.** *For all t, c (and states, suppressed):*

(R1) $[\![\mathsf{rmc}(t,c)]\!] \subseteq [\![\mathsf{expand}(t)]\!]$*;*

(R2) $[\![t]\!] \setminus [\![c]\!] \subseteq [\![\mathsf{rmc}(t,c)]\!]$*.*

Moreover removal is exact *(*$[\![\mathsf{rmc}(t,c)]\!] = [\![\mathsf{expand}(t)]\!] \setminus [\![c]\!]$ *on the clause part) whenever the expansion has no top clause and every same-tag member is a unit, a literal primitive, or full* `boolean` *— assuming additionally the injectivity part (ii) of* ***L-LIT*** *(distinct normalized lexemes denote $\approx$-inequivalent values).*

Proof. (R1): every output member's semantics is a subset of the corresponding input member's ($L \setminus \{\ell_c\} \subseteq L$; the boolean complement is a subset of full boolean; kept members are equal). (R2): let $v \in [\![t]\!] \setminus [\![c]\!]$; by **L-EXP** $v \in [\![\mathsf{expand}(t)]\!]$. If v is in the instance layer or a top clause, it survives unchanged. Otherwise $v \in [\![s]\!]$ for a member s. If $\mathsf{tag}(s) \neq \mathsf{tag}(c)$, s is kept. If s is a unit of c's tag, then $s = c$ (the tag determines the unit) so $[\![s]\!] = [\![c]\!]$, contradicting $v \notin [\![c]\!]$ — vacuous. (A `void`-tagged v with c the `undefined` unit falls under the *different-tag* case: unit(`void`) is kept, and $(\mathsf{void}, \star) \notin [\![c]\!]$ under the singleton unit denotations, so R2 is unaffected.) If s is a literal primitive, $v \approx (\mathsf{tag}(k), \delta_k(\ell))$ for some $\ell \in L$; if $\ell = \ell_c$ then $v \in [\![c]\!]$, contradiction — so $\ell \neq \ell_c$ and ℓ survives the lexeme diff, keeping v in the closure semantics. Note this step needs only that δ is a *function*, not injectivity: R2 is unconditional. If s is full boolean and c a boolean literal, v is the complementary literal. Full other primitives are kept whole. Exactness: a top clause is unchanged while $[\![\top]\!] \setminus [\![c]\!] \subsetneq [\![\top]\!]$, hence its exclusion; in the remaining listed cases each step removes exactly $[\![c]\!]$, using injectivity to ensure no *other* surviving lexeme denotes a member of $[\![c]\!]$. (R1 has no client among the soundness results in this document; its role is precision.) □

Subtracting from top. A top clause is left unchanged: `unknown` minus a constant is still `unknown`. Narrowing `unknown` by `!== null && !== undefined` therefore does not narrow at all — an incompleteness localized to the top-clause case of Definition 39 and to the residue rules (a top clause has no members to filter). Expressing a "non-nullish top" would still require a representation extension.

6 Inference: Substitution Growth and Validation

This section models the substitution state machine exactly and states the soundness architecture as explicit obligations.

6.1 The substitution state machine

Definition 41 (Substitution States)**.** *A substitution state ρ maps type-variable identifiers to* $\mathsf{SVOpen}(t)$ *(a growable lower bound) or* $\mathsf{SVClosed}(t)$ *(a sealed value), and carries a boolean* freeze *(the candidate-freezing flag). Exactly the* ordinary *identifiers (*`TVID`*) are* candidates *(*`TVarID: : isCandidate`*); field-inference (*`TVIDForField`*) and literal-commitment (*`TVIDForLiteralCommitment`*)*

identifiers are not *— the code classifies both as literal-form commitments, and they* are *sealed or widened by a covariant grow. Lemma 43 therefore protects ordinary inference candidates only.*
impl: `TTypeSubst.sk / SubstValue, Subst; TType.sk / TVarID::isCandidate`

Definition 42 (Transitions)**.** $\mathsf{grow}(\alpha, t, var)$ *and* $\mathsf{get}(\alpha, var)$ *act as follows (writing* $\vee$ *for `TType::union`):*

op	*state of* α	*condition*	*effect / result*
$\mathsf{grow}(t, \mathsf{contra})$	*unbound*		$\alpha \mapsto \mathsf{SVOpen}(t)$
	$\mathsf{SVOpen}(t_2)$	$\neg(\mathsf{cand}(\alpha) \wedge \mathsf{freeze})$	$\alpha \mapsto \mathsf{SVOpen}(t_2 \vee t)$
	$\mathsf{SVOpen}(t_2)$	$\mathsf{cand}(\alpha) \wedge \mathsf{freeze}$	*no-op*
	$\mathsf{SVClosed}(_)$		*no-op*
$\mathsf{grow}(t, \mathsf{cov})$	*unbound*		$\alpha \mapsto \mathsf{SVClosed}(t)$
	$\mathsf{SVOpen}(t_2)$	$\mathsf{cand}(\alpha)$	*no-op* (unconditional)
	$\mathsf{SVOpen}(t_2)$	$\neg\mathsf{cand}(\alpha)$	$\alpha \mapsto \mathsf{SVClosed}(t_2 \vee t)$
	$\mathsf{SVClosed}(_)$		*no-op*
$\mathsf{get}(\mathsf{cov})$	*unbound*		*seal* $\alpha \mapsto \mathsf{SVClosed}(\top)$*; return* $\top$
	$\mathsf{SVOpen}(t)$	$\mathsf{cand}(\alpha) \wedge \mathsf{freeze}$	*return* t *(no seal)*
	$\mathsf{SVOpen}(t)$	*otherwise*	*seal* $\alpha \mapsto \mathsf{SVClosed}(t)$*; return* t
	$\mathsf{SVClosed}(t)$		*return* t
$\mathsf{get}(\mathsf{contra})$	*unbound*		*return* $\bot$ *(no write)*
	$\mathsf{SVOpen}(t)$ *or* $\mathsf{SVClosed}(t)$		*return* t

Both get*s then intersect the result with* α*'s constraint types.*
impl: `TTypeSubst.sk / Subst::grow, Subst::get`

Lemma 43 (Determined Candidates Are Never Widened)**.** *For a candidate* α *bound to* $\mathsf{SVOpen}(v)$ *or* $\mathsf{SVClosed}(v)$*: (i) a covariant* grow *never changes* v*; (ii) under* freeze*, no* grow *or* get *changes* v *or its open/closed status; (iii) once* SVClosed*, the binding never changes again.*

Proof. By inspection of Definition 42: (i) the SVOpen/candidate/covariant row is an unconditional no-op and the SVClosed rows are no-ops; (ii) the two freeze-guarded rows are the only remaining writes touching a bound candidate, and both are suppressed; (iii) no transition rewrites an SVClosed binding. □

This is the write-site invariant behind the "covariant checks never widen a determined inference candidate" work (#713, #740): a covariant grow pushes an *expected* type — an upper bound — and the value of a candidate must come from its defining expression, with the comparison against the expected type happening in validation, not by overwriting the candidate.

Write-site audit. The lemma's premise — that Definition 42 captures *every* write to the binding map — has been checked against the source: the only assignments to `Subst.values` are in `grow`, `get`, and the bulk `restore`. External callers of `grow` (array-literal commitment in `TArrayLitTyping.sk`, variadic and utility sites in `TVariadicTyping.sk` / `TypeCheckUtils.sk`)

go through the same transitions; `TVar::setConstraint` (used by intersection) is a *functional* update returning a new `TVar` with an extended constraint set — constraints are occurrence-level, not shared state (registry row SETCONSTRAINT-FUN) — and is accounted for by the constraint intersection in get.

6.2 Growth as pre-seeding; validation decides

Definition 44 (Subtyping Check Architecture)**.** *`isSub`(t_1, t_2) proceeds as: (0) acceptance shortcuts that return before any growth — $t_1 = t_2$, and the reflexive `TSInst` arm (equal name, arguments equal after `peekRaw`), both guidance-free; the `TSInst` arm's `peekRaw` reads an open candidate* without sealing it*, the unsealed-read instance of Obligation 47 (deep dive D); (1)* growTVars$(t_1, t_2,$ contra$)$ *— grow variables of the to-be-grown side t_2 (the implementation's "open" side) from bound material in t_1; (2)* growTVars$(t_2, t_1,$ cov$)$ *— seal variables of t_1; (3) expand both sides under the (now grown) state and validate with `Clauses::isSub`; on failure,* restore *the substitution to its state before (1).* growTVars *itself: collects growable variables of the to-be-grown side (bare, inside intersections if unbound, and behind one-level alias expansion); checks each variable's declared constraints against the bound side* under freeze *(a query, per Lemma 43(ii)), rolling back on failure; grows only the disjuncts of the bound side not already covered by a non-variable branch of that side (each coverage probe is save/restored); finally descends structurally into matching object fields and call/new* return *types.*

impl: `TTypeSubtyping.sk / isSub_, growTVars, collectTVars`

Theorem 45 (Guidance vs. Validation)**.** *Assume the validation obligation **L-SUB**: if `Clauses::isSub` accepts under state ρ, then $[\![t_1]\!]_\rho \subseteq [\![t_2]\!]_\rho$ for the expanded views. Then no imprecision in* growTVars *— growing too little, too much, or in the wrong order — can cause `isSub` to accept an inclusion that is invalid* under the state ρ' that growth produced*. Growth affects only* completeness *(which valid programs are accepted) and the* solutions *chosen for inference variables. (That already-validated judgements are not retroactively weakened by later evolution is a separate matter: Lemma 43 for candidates, Obligations 47 and 48 in general; the failure-path parenthetical in the proof leans on Obligation 48.)*

Proof. Acceptance occurs only in step (3), whose verdict is by **L-SUB** a semantic inclusion *under the state that growth produced.* Growth selects the state; it never bypasses validation. (Failure paths restore the prior state, so failed exploration does not corrupt later checks — see Obligation 48.) □

The force of Theorem 45 is architectural rather than representational. What it makes explicit is that the meaning of *other, already-checked* judgements must not be retroactively weakened by later growth. That is exactly what Lemma 43 secures for candidates, and what the remaining obligations must secure in general:

Obligation 46 (Validation Soundness, **L-SUB**)**.** *`Clauses::isSub` (with its* STypes*, object-branch, field, index, and function strata, including getter/setter variance and the `seenSub`/`tenv.seenSub` cycle caches keyed by substitution version) is sound in the sense of Definition 44. This is the largest unproved component of the document.*

impl: `TTypeSubtyping.sk / Clauses::isSub, STypes::isSub, TObject::isSub, TFields::isSub, TFun::isSub`

Obligation 47 (Monotone Reads Under Growth (registry row OBL-MONOTONE))**.** *If a judgement was validated under ρ and inference later evolves $\rho \to \rho'$ by the transitions of Definition 42, the validated inclusion still holds under ρ'. For* SVClosed *bindings this is Lemma 43(iii); for* SVOpen *bindings of* non-candidates*, unions grow the denotation and the obligation is that every*

consumer read them contravariantly (as lower bounds) until sealed. This obligation is VIOLATED in two witnessed instances — *`subst.seenSub` memos recorded on unsealed candidate bounds inside `freezeCandidates` windows (order-dependent acceptance), and the reflexive `TSInst` arm of Definition 44 step (0), whose `peekRaw` is an exact-value read of an unsealed open candidate (not memo-mediated). The general statement is proved by deep dive E modulo exactly these residues; witnesses and fix directions: `deep-dives/issues/freeze-window-stale-memo-caches.md`, `deep-dives/issues/reflexive-tsinst-fastpath-unsealed-read.md`.*

Obligation 48 (No Commitment on Failure (registry row OBL-RESTORE))**.** *Every failing or exploratory path restores the substitution: `isSub` restores on validation failure; coverage probes and constraint-check failures inside* growTVars *restore; refinement's exploratory intersections run on clones (**L-CLONE**). The stale-cache regression class (#446) is guarded by keying cycle caches with the substitution version.*

7 Worked Examples

7.1 Literal narrowing

A worked case of R-EQ's false branch:

```
function f(x: 1 | 2): 2 {
  if (x === 1) { return 2; }
  return x;  // narrowed to 2
}
```

is accepted: `1 | 2` is prim(num, $\{1, 2\}$); the false branch of R-EQ applies rmc with $c = 1$, and Theorem 40 gives exactly prim(num, $\{2\}$).

7.2 Persisting incompleteness: subtracting from top

`unknown` is the top clause; R-EQ's false branch and the truthiness residues leave top clauses unchanged (Section 5), so `x !== null && x !== undefined` does not narrow `x: unknown`. This remains a representation limitation: there is no form for "all values except these tags".

7.3 Persisting incompleteness: no false-branch `instanceof`

R-INST refines only the true branch; `else` branches keep the unnarrowed type. Sound (identity), incomplete relative to tsc.

7.4 Persisting incompleteness: multi-literal `in` keys

When the key expression's type is a union of literals, R-IN keeps object members unchanged (still dropping primitives on the true branch); per-key case splits are not performed.

7.5 Known unsoundness: string-kind falsiness

The lexeme-level falsiness test classifies a string literal by its stored lexeme (`lexeme == ""`), but the lexeme is not decoded. An ECMA-262 *LineContinuation* — a backslash followed by a line terminator — has string value `""`, so a non-empty backslash-newline lexeme denotes the falsy value

`""` while the test reads it truthy (assumption **L-LIT**(iii)). This is *observable*. Let `L` be such a line-continuation string literal and take `x: L | 0` — both members falsy at runtime. `L` is classified truthy, so the union appears to have exactly one falsy member; the boolean-condition guard passes, the falsy residue of `if (x)` drops `L`, and in the else-branch `const z: 0 = x` is *accepted* while x holds the string `""` at runtime. The canonical control `"" | 0` is correctly rejected as a two-falsy-value condition. The sibling assumption **L-LIT**(ii) fails on the same undecoded-lexeme root: two spellings of one string (e.g. `"a"` and `"\x61"`) are stored as distinct literals, so an equality that holds at runtime is typed as always-false.

8 Assumptions and Proof Obligations

Every axiom this document assumes, with its implementation anchor and a status from a fixed vocabulary: PROVED (established in this document or by inspection of the anchored code), ASSUMED (believed true, undischarged), OPEN (a stated obligation whose discharge is planned), and VIOLATED (a known counterexample exists in the implementation). A status may carry a note. Items whose discharge should be a pinned test say so in the note; wiring those tests is part of the maintenance plan (Appendix B). The obligations of Section 6 (Obligations 46–48) belong to this section's inventory; their current statuses (after the deep-dive series, `deep-dives/README.md`): Obligation 46 partially PROVED with named residues (deep dive D), Obligation 47 VIOLATED in two witnessed instances (see its status note), and Obligation 48 PROVED (deep dive E, with a documented carve-out).

Item	Statement	Status
L-TAG	Every runtime value carries exactly one tag consistent with its static tag.	PROVED
L-UNION	`union` over-approximates set union (all levels).	PROVED
L-INTER	`inter` over-approximates set intersection (the containment clause).	PROVED
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	VIOLATED
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	PROVED
L-MONO	Refinement-time state evolutions preserve membership in already-installed types.	PROVED
L-SEQ	Type denotations used as equality RHSs are $\approx$-closed.	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	OPEN
A-CTOR-CALL	Class-constructor values are typed with call mode None, but constructors implement [[Call]].	VIOLATED
L-TC	Checked expressions inhabit their checked types.	ASSUMED
L-LIT	Literal spellings: (i) δ is a function; (ii) injectivity modulo $\approx$ after normalization; (iii) falsiness-exactness of the lexeme test.	VIOLATED
L-INST	$P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type as instantiated at the narrowing site.	VIOLATED
L-ISARR	$\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$.	PROVED

Continued on next page

Item	Statement	Status
L-LOCAL	Module-level bindings are never narrowed; function-level locals are, including captured ones.	PROVED
L-PURE	Evaluating a condition does not mutate the narrowed locals.	VIOLATED
L-CAPT	Assignments to captured locals are checked against the binding's declared type.	VIOLATED
L-CLONE	A type computed by `inter` under a cloned substitution over-approximates the intersection under the main one.	PROVED
L-MUT	Field writes are governed by `set`-types; get-only membership is sound for snapshot reads.	ASSUMED
L-MUT-ALIAS	A checked write through one mutable view preserves the membership guarantees of every live alias of the same runtime object.	VIOLATED
L-GETSET	Get/set coherence of accessor fields.	OPEN

Details.

L-TAG (PROVED)

Every runtime value carries exactly one tag consistent with its static tag (the runtime counterpart of Lemmas A0/A1, which are proved only inside the model)

Anchor: ECMAScript language types / `typeof` (there is no SKJS code generator; checked programs run on a JS engine)

PROVED (deep dive A: realization map into ECMA-262 §6.1, modulo the environment assumption A-DDA)

L-UNION (PROVED)

`union` over-approximates set union (all levels)

Anchor: `TTypeUnion.sk`

PROVED (deep dive B — in fact exact at every stratum for the membership semantics)

L-INTER (PROVED)

`inter` over-approximates set intersection — the containment clause; the $\bot \Rightarrow$ disjointness clause is the separate row L-INTER-BOT

Anchor: `TTypeInter.sk`

PROVED (deep dive B) at every stratum, using the canonical TVAR-SEM definition for the instance layer

L-INTER-BOT (VIOLATED)

A $\bot$ result from `inter` implies semantic disjointness — the clause every branch-dropping refinement step and L-SUB's covariant variable read (O-GETCOV) consume

Anchor: `TTypeInter.sk`

VIOLATED (deep dive B) in three strata: the `TFun::inter` stratum — incomparable signatures treated as disjoint (issue draft `tfun-inter-incomparable-not-disjoint.md`); the string-kind literal-spelling stratum (undecoded escapes, mediated by L-LIT); and the num-kind beyond-precision spelling stratum — distinct spellings of one double kept distinct, their lexeme intersection a spurious $\bot$ (mediated by L-LIT; issue draft `beyond-precision-numeric-literals.md`). The per-tag `STypes` assembly and within-precision numeric spellings produce only semantic bottoms and are PROVED

L-EXP (PROVED)

Expansion preserves or enlarges semantics; residual instances keep theirs

Anchor: `TType::expand`

PROVED (deep dive C) for the modes refinement uses (`None`/Covariant) — and shown *false* in contravariant mode (unbound reads produce $\perp$), so the axiom is hereby scoped to those modes; all refinement clients verified to use only them. Alias-path cache reuse proved exact (binding-blind normalization); residues: E-NORM, E-ARR (ASSUMED), E-CLASS-SYM, E-GUARD (OPEN) — see deep dive C

L-MONO (PROVED)

Refinement-time state evolutions preserve membership in already-installed types (Axiom 20)

Anchor: `Subst::get`, refinement sites

PROVED (deep dive E, given M1 — semantic monotonicity in variable envelopes — and L-UNION); `setConstraint` is a functional update, so there is no shared-state threat, and the range(Γ) restriction is droppable (see the axiom's remark)

L-SEQ (PROVED)

Type denotations used as equality RHSs are $\approx$-closed

Anchor: `evalVarEquality`

PROVED: by construction for literal/full primitives and objects; the undefined/`void` corner is unreachable — strict equality rejects `void`-typed operands before refinement (empirically pinned, §4.4.1)

L-SUB (OPEN)

`Clauses::isSub` acceptance implies semantic inclusion under the current state

Anchor: `TTypeSubtyping.sk`

split (deep dive D): PROVED — dispatcher architecture, `tenv.seenSub` version-keying (corrected argument), coinductive reduction (via SEM-GFP), primitives, `STypes`, DNF covering, object skeleton, and the field stratum's read direction; conditional — field *set* direction on L-MUT, functions on the E-SIG signature semantics; VIOLATED — `subst.seenSub` inside freeze windows (witnessed; issue draft `freeze-window-stale-memo-caches.md`), the `isSubOptTFuns` `None`-mode arm (F-CALL) and the never-compared array brand (F-BRAND), both facets of the `call=None` dichotomy

A-CTOR-CALL (VIOLATED)

Class-constructor values are typed with call mode `None`, but constructors implement [[Call]] — the refinement side of the `call=None` dichotomy (Definition Signature Modes)

Anchor: `newStaticObject`; `evalTypeofEquality`

VIOLATED (witnessed: a `string`-typed function returns a class at runtime through the `typeof "function"` branch drop; deep dive A; issue draft `class-constructor-call-none-typeof.md`)

L-TC (ASSUMED)

Checked expressions inhabit their checked types (used for the RHS of R-EQ and field reads)

Anchor: `TypeCheckExpr.sk`

ASSUMED (axiom here; a full development would discharge it by mutual induction with a preservation theorem, of which the branch-soundness lemmas are the condition case)

L-LIT (VIOLATED)

Three claims about literal spellings: (i) δ is a function — definitional, used by R2; (ii) *injectivity modulo* $\approx$ after normalization — needed only for *exactness* claims; (iii) *falsiness-exactness*: the

lexeme test `isFalsy` agrees with Definition 6 — needed by the residue lemma

Anchor: `normalizeJSNumber/normalizeJSBigInt`; `isFalsy`; `unquoteString`

(i) PROVED (definitional). For num/bigint, (ii)/(iii) are PROVED for *within-precision* literals: normalized to canonical form at parse time (`normalizeJSNumber/normalizeJSBigInt`), so on that range zero has one spelling per kind and injectivity holds. For *beyond-precision* numeric literals — those failing the round-trip `String(Number(s))` = s: 16–17-digit decimals, radix forms $> 2^{53}$, over/underflow exponent forms — (ii)/(iii) are VIOLATED: distinct spellings of one double are kept distinct, so `===` narrowing yields a spurious `never` (ii), and the normalizer can emit a zero-denoting lexeme (`1e-400`, underflow to +0) classified truthy (iii) — both witnessed end-to-end (issue draft `beyond-precision-numeric-literals.md`). For the *string* kind, (ii)/(iii) are VIOLATED: escapes are stored undecoded (`unquoteString`; issue draft `string-literal-escapes-undecoded.md`), so distinct spellings of one string are treated as distinct (ii), and a line-continuation lexeme denotes `""` yet is classified truthy (iii) — both witnessed end-to-end. All other results are unconditional

L-INST (VIOLATED)

$P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type as instantiated at the narrowing site

Anchor: `evalInstanceof`

decomposed (deep dive A): conditionally PROVED modulo A-HASINST — VIOLATED, tsc-parity (issue draft `symbol-hasinstance-narrowing.md`) — and A-PROTO (ASSUMED, enforced for checked code by stub omissions). Note that `makeTArgs` yields fresh unbound inference candidates; under canonical TVAR-SEM they denote empty lower envelopes, so L-INST fails when the narrowed type is installed, and a later covariant use exposes the failure by sealing the argument from context (witnessed; issue draft `instanceof-type-argument-sealing.md`)

L-ISARR (PROVED)

$\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$

Anchor: `evalIsArray`

PROVED (deep dive A: spec-level `IsArray` analysis + verification that the `__SKJS_isArray` brand is stripped into the `isArray` component, never a required field) modulo A-PROXY

L-LOCAL (PROVED)

Module-level bindings are never narrowed (they resolve through `defMap`, not the scope chain, so the primitive's early return fires). Function-level locals *are* narrowed — including locals of enclosing functions reached from inside a closure, since scope lookup walks the whole chain

Anchor: `setLocalFilterMap`; `TypeCheckScope.sk`

PROVED (by code inspection) for the module-level half; the captured-local half sharpens L-PURE; a pinned test is recommended. Narrowing module-level bindings is a tracked future enhancement (issue #585, blocked on immutability)

L-PURE (VIOLATED)

Evaluating a condition does not mutate the narrowed locals

Anchor: —

VIOLATED (empirically pinned): the checker narrows across a call that assigns a captured local and accepts the resulting unsound program (`if (x !== null && mut())` with `mut` setting `x = null`) — the same unsoundness tsc accepts. Tracked as issues #597 (side-effecting call between test and use) and #599 (captured local mutated via a lambda). The one-step scoping of the branch-soundness theorems (§1) is therefore a real boundary, not caution. Deep dive F specifies the fix: a layered invalidation policy (P1–P4) with a preservation lemma replacing the one-step scoping

L-CAPT (VIOLATED)
Assignments to captured locals are checked against the binding's declared type (an implicit premise of L-TC for closure bodies)

Anchor: `Scope::set`; `TypeCheckExpr.sk` (lambda checking)

VIOLATED (witnessed: a captured-local write is never checked at all — arbitrary type confusion with no refinement involved; distinct root cause from #597–#599; issue draft `captured-local-writes-unchecked.md`). Sibling: closure bodies read enclosing narrowings that are stale at invocation (the read-in dual of #599; issue draft `stale-closure-reads-narrowing.md`)

L-CLONE (PROVED)
A type computed by `inter` under a cloned substitution over-approximates the intersection when later read under the main one

Anchor: `evalVarEquality`

PROVED (deep dive E, fork-state bridging, modulo the fork-state form of L-INTER); the clone id-collision residue CL-1 was verified *unreachable* (no `gen*` call site in the clone-compute-install tree) and should be guarded by the recommended CI assertion

L-MUT (ASSUMED)
Field writes are governed by `set`-types via subtyping; membership semantics (get-only) is sound for the snapshot reads refinement performs

Anchor: `TField`

ASSUMED; full mutation model out of scope. The tracked instance is issue #598: field reassignment inside a refined branch does not re-widen the narrowed field, invalidating R-EqField/R-TruthyField results past the write; deep dive F specifies the re-widening rule (W-Field)

L-MUT-ALIAS (VIOLATED)
Cross-view mutation coherence: a checked write through one mutable view preserves the membership guarantees of every live alias of the same runtime object

Anchor: `evalInstanceof`; indexed writes

VIOLATED (deep dive H) at two witnessed sites — minted views (`instanceof Array`; issue draft `instanceof-array-mutable-top-view.md`) and narrowing-admitted aliases (issue draft `narrowed-alias-stale-write.md`); declaration-only aliasing is coherent, conditional on L-SUB-FIELDS-SET

L-GETSET (OPEN)
Get/set coherence of accessor fields — the prerequisite for the sound accessor variant of deep dive F's W-Field re-widening rule

Anchor: `TField` (get/set split)

OPEN (deep dive F)

Series-introduced items. The deep-dive series introduces further named assumptions and obligations (interface facts its proofs consume, and residues its audits leave). They are registry entries like the rows above and carry the same status vocabulary:

Item	Statement	Status
OBL-MONOTONE	Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.	VIOLATED

Continued on next page

Item	Statement	Status
OBL-RESTORE	No Commitment on Failure: failing and exploratory paths restore all relevant state.	PROVED
MR-1	No `subst.seenSub` memo recorded under `freezeCandidates` may outlive growth of an open candidate readable from its left side.	VIOLATED
C-FREEZE-CACHE	Isolated attribution of `seenInter` freeze-window staleness (mechanism active; shared witness with MR-1).	OPEN
CL-1	No fresh variable generated on a clone occurs in a type installed against the main substitution.	PROVED
M1	Semantic monotonicity of $[\![\cdot]\!]$ in variable envelopes (a property of the get-only, signature-opaque semantics).	ASSUMED
I-F	Fork-state form of L-INTER: the containments hold from a forked substitution state (consumed by L-CLONE's proof).	ASSUMED
RB-1	Maintenance condition: `isSub` callers discarding errors are save/restore-wrapped or unreachable by uncommitted array literals.	OPEN
E-NORM	Named-type normalization is semantics-preserving (interface consumed by expansion).	ASSUMED
E-ARR	Delayed array-literal forms keep their semantics under expansion.	ASSUMED
E-CLASS-SYM	The class path of expansion is binding-blind (or its binding reads are harmless).	OPEN
E-GUARD	Recursion-guard identity expansions are never cached into long-lived substitutions.	OPEN
SEM-GFP	The set-theoretic semantics is the greatest solution of the recursive membership equations.	PROVED
E-SIG	Behavioral call-soundness semantics for function signatures (required for function subtyping to be a soundness statement).	ASSUMED
A-HASH	`TType::compare` equality is trustworthy (XXHash-only fast paths).	ASSUMED
A-PROTO	No prototype surgery / Proxy GetPrototypeOf traps reach checked code (enforced by stub omissions).	ASSUMED
A-DDA	No [[IsHTMLDDA]] object (`document.all`) reaches checked code (vacuous on Node).	ASSUMED
A-PROXY	No Proxy exotic object reaches checked code.	ASSUMED
A-HASINST	No non-default `Symbol.hasInstance` on narrowed `instanceof` RHS classes.	VIOLATED
TVAR-SEM	A type-variable occurrence denotes its substitution envelope intersected with its occurrence constraints; an unbound envelope is empty.	PROVED
SET-SEM	A view authorizes exactly the writes its resolved `set`-types accept; a readonly field authorizes nothing.	PROVED

Continued on next page

Item	Statement	Status
L-SUB-FIELDS-SET	`isSub`'s field arms preserve write capability downward: acceptance implies each target field's `set`-type is below the source's, and readonly fields acquire no writers.	OPEN
O-GETCOV	Soundness of the covariant variable read in subtyping (inherits L-INTER's violated $\bot$-clause).	OPEN
O-GETCONTRA	Soundness of the contravariant variable read in subtyping.	OPEN
O-ARRAYLIT	Soundness of the array-literal decision procedure inside subtyping.	OPEN
F-CALL	The `isSubOptTFuns None`-mode arm reads `call=None` as unknown callability (vs. refinement's definitely-not-callable).	VIOLATED
F-BRAND	Subtyping never compares the array brand, though refinement narrows on it.	VIOLATED
SETCONSTRAINT-FUN	`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.	PROVED
E-TRANSPORT	Monotone-reads transport: an inclusion validated under ρ survives later legal transitions on the same timeline, modulo MR-1.	PROVED
TENV-EXPANDINTER	The `tenv.expandInter` mode flag is path-scoped, and both intersection forms it selects (expanded and deferred `unexpandedInter`) satisfy the L-INTER containment.	OPEN
L-SUB-IDXNAME	A numeric-named field is checked against both a number and a string index when the supertype has both (soundness of the width lemma's numeric-named case).	OPEN
A-INTERSEM	$[\![\mathsf{TInter}(\bar{t})]\!] \subseteq \bigcap [\![t]\!]$ — the instance-layer upper bound the dispatcher's fast paths assume.	ASSUMED
E-GUARD-PLACEHOLDER	An un-annotated ambient-class field placeholder must not head-expand to $\bot$ (the case E-GUARD's recursion-guard audit did not cover).	VIOLATED
ACCESSOR-METHOD	Accessor (`get`/`set`) members must be typed as fields, not methods; today a getter is a call signature, so `c.p()` type-checks and throws.	VIOLATED
L-SUB-TSINST	The reflexive `TSInst` fast path in `isSub` (same name, arguments equal after `peekRaw`) is sound.	VIOLATED

Details.

OBL-MONOTONE (VIOLATED)

Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.

Anchor: `Subst::get`; `TType::isSub`

VIOLATED in two witnessed instances — the freeze-window memo staleness MR-1 and the unsealed `peekRaw` read of the reflexive `TSInst` arm (L-SUB-TSINST, not memo-mediated); the general transport statement is the separate PROVED row E-TRANSPORT

OBL-RESTORE (PROVED)

No Commitment on Failure: failing and exploratory paths restore all relevant state.

Anchor: `SavedSubst`; save/restore sites

PROVED (deep dive E: 14-row save/restore audit; `SavedSubst` completeness; array-literal carve-out sound under RB-1)

MR-1 (VIOLATED)

No `subst.seenSub` memo recorded under `freezeCandidates` may outlive growth of an open candidate readable from its left side.

Anchor: `Subst::get` (freeze arm); `TType::isSub` memo

VIOLATED, witnessed end-to-end (order-dependent acceptance; deep dive E; issue draft `freeze-window-stale-memo-caches.md`)

C-FREEZE-CACHE (OPEN)

Isolated attribution of `seenInter` freeze-window staleness (mechanism active; shared witness with MR-1).

Anchor: `seenInter`; `TType::inter`

OPEN for the isolated statement; mechanism demonstrated active (deep dive B); the MR-1 fix covers both memos

CL-1 (PROVED)

No fresh variable generated on a clone occurs in a type installed against the main substitution.

Anchor: `Subst::clone`, `Subst::gen`

discharged (deep dive E): unreachable — no `gen*` call site in the clone-compute-install tree; guard with the recommended CI assertion

M1 (ASSUMED)

Semantic monotonicity of $[\![\cdot]\!]$ in variable envelopes (a property of the get-only, signature-opaque semantics).

Anchor: main paper semantics

ASSUMED (deep dive E; provable by structural induction in the main paper)

I-F (ASSUMED)

Fork-state form of L-INTER: the containments hold from a forked substitution state (consumed by L-CLONE's proof).

Anchor: `TTypeInter.sk`

ASSUMED (deep dives B and E); folds into L-INTER's discharge

RB-1 (OPEN)

Maintenance condition: `isSub` callers discarding errors are save/restore-wrapped or unreachable by uncommitted array literals.

Anchor: `ArrayLitTyping`; `isSub` probe sites

OPEN maintenance condition (deep dive E); CI grep recommended

E-NORM (ASSUMED)

Named-type normalization is semantics-preserving (interface consumed by expansion).

Anchor: `TEnv::normalize`

ASSUMED (deep dive C)

E-ARR (ASSUMED)

Delayed array-literal forms keep their semantics under expansion.

Anchor: `TArrayLit`

ASSUMED (deep dive C)

E-CLASS-SYM (OPEN)

The class path of expansion is binding-blind (or its binding reads are harmless).

Anchor: `newObject/addFields`

OPEN (deep dive C; proof and counterexample both outstanding)

E-GUARD (OPEN)

Recursion-guard identity expansions are never cached into long-lived substitutions.

Anchor: `seenExpansion`; recursion guards

OPEN (deep dive C); cache-write invariant + pinned test recommended

SEM-GFP (PROVED)

Greatest-fixpoint reading: $[\![\cdot]\!]$ is the greatest solution of the recursive membership equations.

Anchor: main paper object semantics

PROVED by definition in the main paper (every recursive occurrence in the membership conditions is positive, so the greatest solution exists); deep dive D supplies the stratified characterization used by the coinductive reduction

E-SIG (ASSUMED)

Behavioral call-soundness semantics for function signatures (required for function subtyping to be a soundness statement).

Anchor: main paper object semantics (extension)

ASSUMED (deep dive D; proposed extension of the main paper's object semantics)

A-HASH (ASSUMED)

`TType::compare` equality is trustworthy (XXHash-only fast paths).

Anchor: `TType` hashing

ASSUMED (deep dive D)

A-PROTO (ASSUMED)

No prototype surgery / Proxy GetPrototypeOf traps reach checked code (enforced by stub omissions).

Anchor: import-stubs

ASSUMED (deep dive A; enforced for checked code by stub omissions)

A-DDA (ASSUMED)

No [[IsHTMLDDA]] object (`document.all`) reaches checked code (vacuous on Node).

Anchor: environment

ASSUMED (deep dive A)

A-PROXY (ASSUMED)

No Proxy exotic object reaches checked code.

Anchor: environment

ASSUMED (deep dive A)

A-HASINST (VIOLATED)

No non-default `Symbol.hasInstance` on narrowed `instanceof` RHS classes.

Anchor: `isInstanceofRhs`

VIOLATED, tsc-parity (deep dive A; issue draft `symbol-hasinstance-narrowing.md`)

TVAR-SEM (PROVED)

Canonical `TVar` denotation: an occurrence denotes its substitution envelope intersected with its occurrence constraints; an unbound envelope is empty.

Anchor: `Subst::get/peek`

PROVED by definition in the main paper and matched to `Subst::get`: missing bindings read as bottom contravariantly and seal to top before a covariant expansion; occurrence constraints are intersected after either read

SET-SEM (PROVED)

Write authorization: a view authorizes the write of v at key p iff v inhabits the resolved set-type of p (intersected named-field set-types, else the covering index's); readonly ($\mathsf{set} = \bot$) authorizes nothing; call-mediated writes are E-SIG's

Anchor: `TFields::resolveFieldType` (`isLValue`)

PROVED by definition in the main paper and matched to the checker's write path: assignment checks the right-hand side against the resolved set-type by subtyping

L-SUB-FIELDS-SET (OPEN)

Write-capability preservation: field acceptance requires $f_2.\mathsf{set} \leq f_1.\mathsf{set}$ (contravariant) and the read-only gate, so under SET-SEM an admitted view authorizes only writes the source view authorizes

Anchor: `TFields::isSub_`

OPEN (conditional): the syntactic checks are verified at every field arm (deep dive D, write-direction lemma); the SET-SEM containment they induce is conditional on the recursive `isSub` verdicts, exactly as the read direction is inside L-SUB

O-GETCOV (OPEN)

Soundness of the covariant variable read in subtyping (inherits L-INTER's violated $\bot$-clause).

Anchor: `Subst::get` (covariant); `expandToClauses`

OPEN (deep dive D): the covariant variable read applies `Subst::get`'s constraint-intersection loop, whose soundness rests on the `L-INTER-BOT` bottoms; its live strata (incomparable signatures; string-spelling; beyond-precision num-spelling) leave the read unproven. On L-SUB's critical path

O-GETCONTRA (OPEN)

Soundness of the contravariant variable read in subtyping.

Anchor: `Subst::get` (contravariant)

OPEN (deep dive D)

O-ARRAYLIT (OPEN)

Soundness of the array-literal decision procedure inside subtyping.

Anchor: `ArrayLitTyping::subtyping`

OPEN (deep dive D; delegated)

F-CALL (VIOLATED)

The `isSubOptTFuns None`-mode arm reads `call=None` as unknown callability (vs. refinement's definitely-not-callable).

Anchor: `isSubOptTFuns`

VIOLATED, witnessed end-to-end (deep dive D): a callable assigned to {} then narrowed by `typeof` to `never`; issue draft `class-constructor-call-none-typeof.md`

F-BRAND (VIOLATED)

Subtyping never compares the array brand, though refinement narrows on it.

Anchor: `TFields::isSub_`

VIOLATED by code-level derivation (deep dive D; composite witness sketch; no end-to-end run)

SETCONSTRAINT-FUN (PROVED)

`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.

Anchor: `TVar::setConstraint`

PROVED (deep dive E, by code inspection: frozen class, functional update, both call sites use the return value)

E-TRANSPORT (PROVED)

Monotone-reads transport: an inclusion validated under ρ survives later legal transitions on the same timeline, modulo MR-1.

Anchor: `Subst` transitions

PROVED (deep dive E, the monotone-reads theorem, given M1 and the MR/BR audit; residue MR-1)

TENV-EXPANDINTER (OPEN)

The `tenv.expandInter` mode flag is path-scoped, and both intersection forms it selects (expanded and deferred `unexpandedInter`) satisfy the L-INTER containment.

Anchor: `TEnv.expandInter`; `TInter::expand`

OPEN — found by external review: the flag is absent from deep dive E's unsaved-state inventory; path-scoping should follow the frozen-TEnv threading argument, and the deferred form needs deep dive B's containment coverage

L-SUB-IDXNAME (OPEN)

A numeric-named field is checked against both a number and a string index when the supertype has both (soundness of the width lemma's numeric-named case).

Anchor: `indexForFieldName`; `TFields::isSub_`

OPEN — found by external review: the width lemma checks a numeric-named sub field against only one super index (`indexForFieldName` returns a single index), while covering semantics needs both; SKJS enforces neither TS2411 nor TS2413. The subtyping over-acceptance is real but *not* end-to-end unsound at the pin — reads of numeric-string keys stay conservative (`string|number`), masking it, as verified against the binary; a validator-completeness gap, not a witnessed unsoundness. Cited at deep dive D's width lemma as a named residual

A-INTERSEM (ASSUMED)

$[\![\mathsf{TInter}(\bar{t})]\!] \subseteq \bigcap [\![t]\!]$ — the instance-layer upper bound the dispatcher's fast paths assume.

Anchor: `TInter::expand`

ASSUMED (deep dive D; the instance-layer counterpart of deep dive C's `TInter` expansion — an upper bound on the denotation, distinct from L-EXP's over-approximation direction)

E-GUARD-PLACEHOLDER (VIOLATED)

An un-annotated ambient-class field placeholder must not head-expand to $\bot$ (the case E-GUARD's recursion-guard audit did not cover).

Anchor: `instantiate` (missing-annotation return); `expandToVectors`

VIOLATED, witnessed end-to-end (deep dive C): the missing-annotation identity return fires with `seenInstantiate` empty, the identity is dropped by the expansion seen-set, and the field reads as $\bot$ (accepted into `never`); issue draft `ambient-unannotated-field-bottom.md`. Sibling of the recursion-guard E-GUARD row, which is empirically safe

ACCESSOR-METHOD (VIOLATED)

Accessor (`get/set`) members must be typed as fields, not methods; today a getter is a call signature, so `c.p()` type-checks and throws.

Anchor: `isGetSet` (parser-only); member typing

VIOLATED, witnessed end-to-end (SKJS-specific; tsc rejects with TS6234): the parser's `isGetSet` marker is never read by the checker, so a getter `get p(): T` is typed as a method `p(): T`. `c.p()` is accepted and throws at runtime (`c.p` is not a function). Issue draft `getter-typed-as-method.md`. Adjacent to F's accessor caveat and L-GETSET, but a distinct (member-typing) hole

L-SUB-TSINST (VIOLATED)

The reflexive `TSInst` fast path in `isSub` (same name, arguments equal after `peekRaw`) is sound.

Anchor: `TType::isSub` (TSInst reflexive arm)

VIOLATED (deep dive D): the inclusion at the verdict state holds for unconstrained bindings, but the arm reads an unsealed `SVOpen` candidate via `peekRaw` without the covariant seal the slow path performs, so a later contravariant grow widens the binding and reads through aliases break — witnessed end-to-end (a value typed `number` holds a string; issue draft `reflexive-tsinst-fastpath-unsealed-read.md`). For constrained variables the equality claim also fails as stated: the occurrence denotes $[\![t_2]\!]_\rho \cap \bigcap [\![C]\!]$, not $[\![t_2]\!]_\rho$

In summary, after the deep-dive series and its adversarial verification (series map in `deep-dives/README.md`; the authoritative per-item statuses are the tables above, from which this grouping is generated):

- *Proved*: L-TAG; L-UNION; L-INTER's containment (all strata); L-EXP (mode-scoped); L-MONO; L-SEQ; the majority strata of L-SUB; L-ISARR; L-CLONE; OBL-RESTORE; SEM-GFP; TVAR-SEM; SET-SEM.
- *Assumed*: L-TC (the capstone); L-MUT.
- *Open*: L-SUB's conditional strata; L-GETSET; the isolated `seenInter` attribution (C-FREEZE-CACHE); E-CLASS-SYM; E-GUARD; O-GETCOV, O-GETCONTRA, O-ARRAYLIT; TENV-EXPANDINTER; L-SUB-IDXNAME.
- *Violated, with an executed witness* (each with a tracked issue or an issue draft under `deep-dives/issues/`): the $\bot \Rightarrow$disjoint clause L-INTER-BOT (`TFun` + string/num spellings); A-CTOR-CALL; L-LIT(ii)/(iii) string-kind (undecoded escapes) + num beyond-precision spellings; L-INST's A-HASINST and fresh-argument representation; L-PURE (#597/#599) with the L-MUT instance #598; L-CAPT and its stale-read sibling; cross-view mutation coherence L-MUT-ALIAS (minted `Array`$\langle\top\rangle$ views; narrowing-admitted aliases); the two witnessed instances of OBL-MONOTONE (MR-1; the reflexive-`TSInst` read); L-SUB's `call=None` subtyping arm (F-CALL); the ambient-field $\bot$-collapse (E-GUARD-PLACEHOLDER); accessor members typed as methods (ACCESSOR-METHOD); L-SUB-TSINST (unsealed `peekRaw` read in the reflexive `TSInst` arm).
- *Violated by code-level derivation* (no end-to-end run yet): the never-compared array brand (F-BRAND).

The violations remain local to leaf assumptions: every fix restores its assumption without changing a theorem statement or proof in this document. Two design decisions gate several fixes and are recorded as fundamental holes rather than bugs: the `call=None` reading dichotomy and the

refinement-invalidation policy (deep dive F). Known unsoundness here is enumerated, witnessed, and quarantined — future work, not architectural debt.

None recorded.

A Implementation Correspondence

Pinned to `cf5a6bc7`.

Model	Implementation
Tags (9 tags)	`STypeTag` (TType.sk)
unit, prim, object	`TPrimUnit`, `TPrimValue`, `TObject` (TType.sk)
literal payloads	`PrimLiterals`
object branches B, fields	`ArraySet<TFields>`, `TField`
sig modes None/(opt, σ)	`TFields.call/new : ?(Bool, TFuns)`
array brand	`TFields.isArray : ?Bool`
phantom fields	`FAPhantom`
STypes	`STypes` (tag-keyed map)
$\langle V \mid S \rangle$, Clauses, TType	`Clause`, `Clauses`, `TType`
refine^{g,d}	`setLocalFilterMap` + `TType::mapSTypes`
master list dispatch	`evalCond` (TypeCheckRefine.sk)
rmc	`TType::removeConstant`
tres$_b$, vres$_b$	`truthinessResidue`, `truthinessValueResidue`
scope merge $\sqcup$	`TCEnv::mergeScope`
ρ, transitions	`Subst`, `SVOpen`/`SVClosed`, `grow`/`get` (TTypeSubst.sk)
freeze	`Subst.freezeCandidates`
growth pre-pass	`growTVars` (TTypeSubtyping.sk)
validation	`Clauses::isSub`

B Maintenance

An advisory maintenance rule ("when the implementation adds a new refinement, add it to this list") does not survive contact with development. The replacement is enforced coupling; the parts marked *in place* are implemented by `docs/tools/docs.py` (`gen` / `check` / `build`):

1. **The registry** (*in place*). Statuses live only in `docs/ledger.yaml`; the tables of Section 8, their summary, and the deep dives' rows tables are generated from it (`docs/generated/`). A status change is a one-line registry edit; no document states another document's status.

2. **Consistency checks** (*in place*). The `check` command verifies: registry integrity (status vocabulary, anchor files exist, issue references resolve); every issue draft is referenced; no temporal/history markers in any document; no numeric cross-document references (deep dives must cite main-paper items by *name*); every document's date goes through the shared pin (`docs/shared/pin.tex`).

3. **Local builds** (*in place*). The `build` command compiles every document (two `pdflatex` passes), keeps each `.pdf` next to its source, fails on errors or unresolved references, and keeps auxiliary files in the git-ignored `docs/.texbuild/` for warm incremental recompilation.

4. **Anchor cross-check** (*pending*). Each dispatch arm of `evalCond` carries a comment `// paper: R-...` naming its rule; the check verifies both directions (every arm has a rule; every master-list rule has a live arm). Existence, not correctness — semantic drift within an arm still requires review, but silent additions and removals become failures.

5. **Witness repros** (*in place for issue witnesses*). Each executed unsoundness witness is a runnable `deep-dives/issues/repros/*.ts` with a header verdict; `docs.py repro` runs each against the pinned checker and asserts it, and `check` enforces that every repro is referenced by an issue draft and vice versa. A repro that flips verdict when its fix lands fails, forcing a documentation update in the same change. Still *pending*: expect-error pins for the incompleteness worked examples in the test suite proper.

6. **Pinned revision.** `docs/shared/pin.tex` names the modeled commit once, for every document. A change that trips any check updates the pin as part of restoring it.

The review protocol for this corpus — scopes and checks for reviewing this paper, the deep dives, and the issue drafts — lives in `docs/README.md`.